

Universidad del Valle de Guatemala

Ingeniería de Software

Sección 30

Ing. Lynette García



Corte 3 - Proyecto “Sistema administrador para cadena de farmacias San Gabriel”

Josué Antonio Isaac García Barrera – 24918

José Manuel Sanchez Hernández - 24092

José Alberto Abril Suchite - 24585

Pablo André Orellana Mijangos - 20555

Andrés Esteban Ismalej González - 24005

Guatemala, 18 de marzo del 2026

1. Resumen

En este trabajo se presenta el proceso para definir como tema de proyecto el desarrollo de un sistema administrador especializado para farmacias minoristas, con el que se busca resolver problemas operativos en la cadena "San Gabriel". Ya que debido a la falta de una herramienta centralizada se generan ineficiencias en la administración del inventario, el control financiero y la toma de decisiones, lo que afecta la productividad del negocio.

La idea del proyecto surge por la necesidad de modernizar procesos que actualmente dependen de métodos manuales y sistemas genéricos, lo que provoca errores frecuentes, pérdida de información y descoordinación entre sucursales. Implementar una solución tecnológica adaptada a la farmacia es clave para garantizar que se tenga un control sobre ella. Para ello, se plantearon los siguientes objetivos:

1. Reducir el tiempo de registro de nuevos productos en el sistema a menos de 10 minutos por medicamento, permitiendo el ingreso de datos como nombre comercial, nombre genérico, casa médica, presentación, fecha de vencimiento y precio, disminuyendo la dependencia de registros manuales.
2. Disminuir los errores de inventario en al menos un 70 % durante las primeras 3 semanas de uso, mediante la automatización del control de existencias y la eliminación de registros duplicados entre sucursales.
3. Permitir la búsqueda de medicamentos en menos de 5 segundos, utilizando filtros por nombre comercial, nombre genérico y casa médica, mejorando la atención al cliente en el mostrador.
4. Generar reportes de ventas consolidados en tiempo real para las tres sucursales, con un retraso máximo de 5 segundos, permitiendo al administrador visualizar ingresos diarios, semanales y mensuales.
5. Implementar alertas automáticas de vencimiento que notifiquen productos con 30, 15 y 7 días de anticipación, reduciendo la pérdida de medicamentos vencidos en al menos un 50 % en un lapso de 1-6 meses.
6. Centralizar el historial de precios de los medicamentos, permitiendo consultar cambios realizados en un período máximo de 3 clics, facilitando auditorías y toma de decisiones financieras.
7. Reducir el tiempo de cierre de caja diario a menos de 10 minutos por sucursal, integrando las ventas en efectivo.

2. Introducción

La entidad a la que se dirige la solución es la cadena de farmacias “San Gabriel” guatemalteca dedicada a la compra y venta minorista de productos farmacéuticos, con presencia en la región de Baja Verapaz, Guatemala. Actualmente, la organización cuenta con tres sucursales activas, las cuales atienden de manera directa a la población local, ofreciendo medicamentos y productos relacionados con la salud, desempeñando un rol relevante en el acceso a servicios farmacéuticos de la región.

En relación, se detectó el problema de gestión de inventario, ventas y carencia de estadística de ventas, gracias al contacto con el doctor Mateo Ismalej Chen, quien nos comunicó su situación y sus principales procesos como lo son, el ingreso de productos nuevos al programa actual “Eleventa” en donde no se pueden ingresar datos comunes de los medicamentos, carece de una búsqueda fácil de los medicamentos según filtros como casa médica/nombre comercial/nombre genérico y nula visualización de estadísticas de ventas por farmacia, actualizadas en tiempo real. Es así como el proyecto pretende ser una herramienta para un control automatizado de inventario, que permita a los vendedores dentro de las farmacias llevar el control mucho más minucioso de los medicamentos y no requerir a procesos manuales que no suelen ser fiables, así como proporcionarle una herramienta al dueño que unifique las estadísticas de ventas de sus 3 actuales sucursales y se las proporcione en un formato atractivo e intuitivo para la gestión y toma de decisiones financieras.

Objetivo general:

Realizar el análisis, diseño y planificación del proyecto para la cadena de farmacias San Gabriel, documentando su arquitectura, funcionalidades y selección tecnológica, con el fin de establecer una base sólida y detallada que guíe su futura implementación y asegure la cobertura de los requisitos del negocio.

Objetivos específicos:

1. Desarrollar versiones de prototipos de interfaz de usuario, incorporando la retroalimentación de potenciales usuarios para refinar el diseño y las funcionalidades del sistema, asegurando que la solución propuesta se centre en las necesidades reales de los actores de la farmacia.
2. Definir una lista detallada de requisitos funcionales, vinculándolos a las historias de usuario correspondientes. Modelar la estructura del sistema mediante diagramas de clases, de paquetes y de clases persistentes, garantizando un diseño con alta cohesión y bajo acoplamiento.
3. Diseñar el modelo de datos relacional: Elaborar el diagrama entidad-relación (DER) para la base de datos, describiendo las entidades necesarias para la persistencia de la información y justificando la elección de un modelo relacional para garantizar la integridad de los datos transaccionales.
4. Evaluar y seleccionar, basándose en los requisitos no funcionales y las estimaciones de crecimiento del sistema, las tecnologías específicas para el

backend, frontend y base de datos. Justificar la elección de la arquitectura en capas y los patrones de diseño propuestos para asegurar la escalabilidad, el rendimiento y la mantenibilidad del sistema.

- 5. Planificar y gestionar las actividades del proyecto: Desglosar el trabajo del corte en tareas asignadas a los miembros del equipo, y elaborar un informe de gestión de tiempo que permita reflexionar sobre el desempeño grupal e identificar aspectos positivos y áreas de mejora en la organización del trabajo.

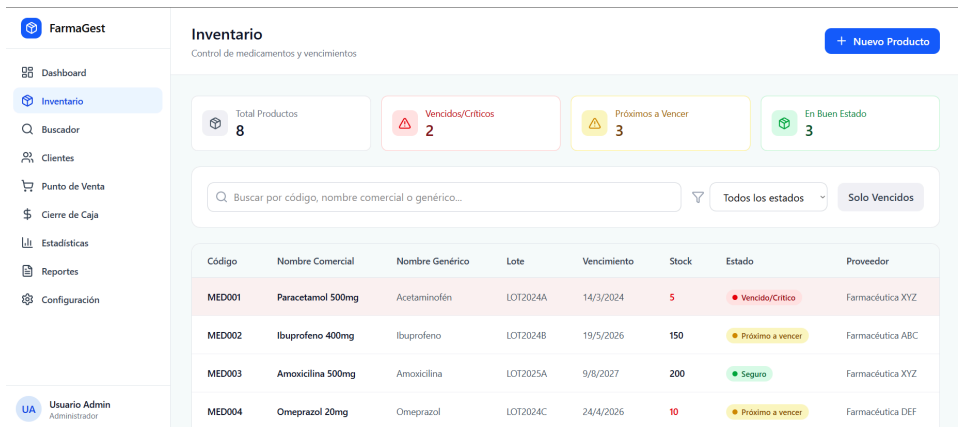
3. Design Thinking

a. Prototipos

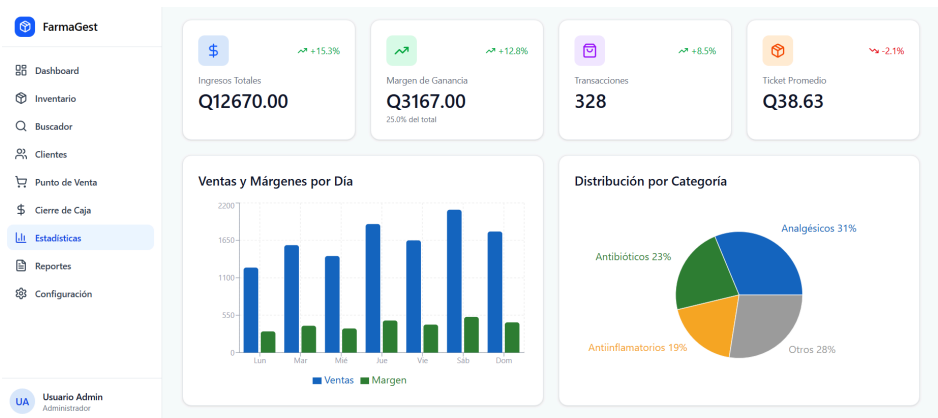
i. Prototipos para diferentes historias de usuario

Primer serie de prototipos v1:

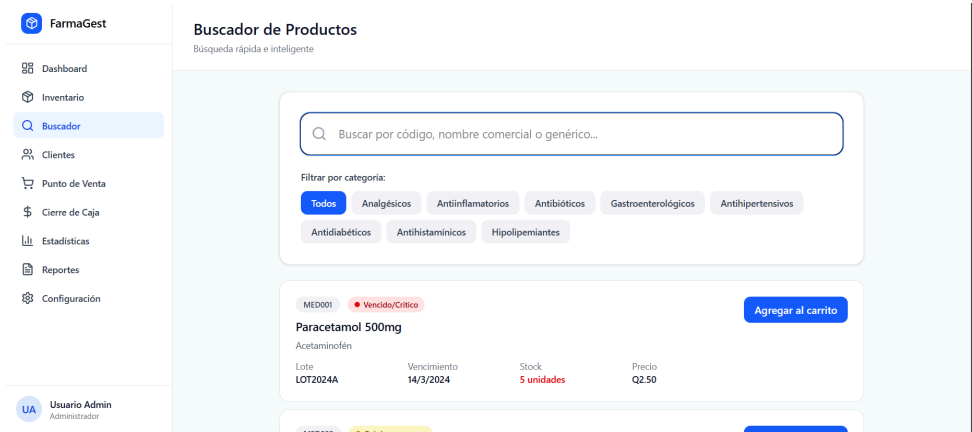
Dashboard: información preliminar sobre el estatus de los productos y manejo de cuentas en tiempo real.



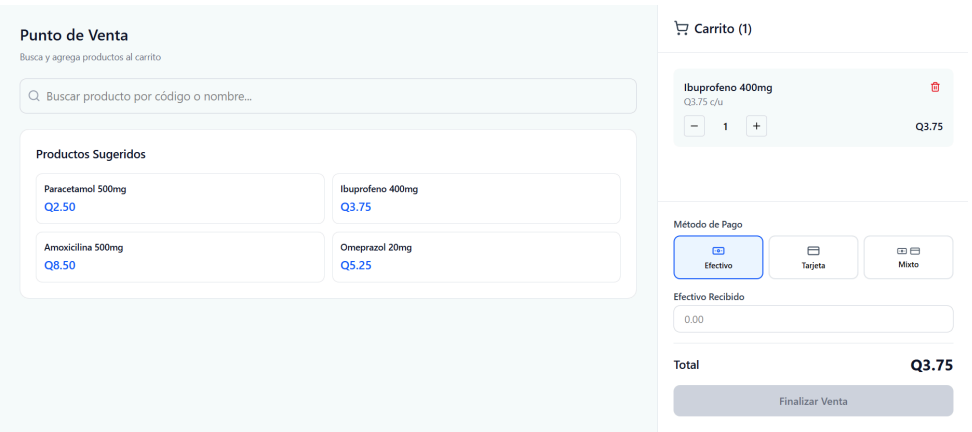
Estadísticas: mejor control de economico de ventas y óptimo despliegue de estadística básica para la toma de desiciones.



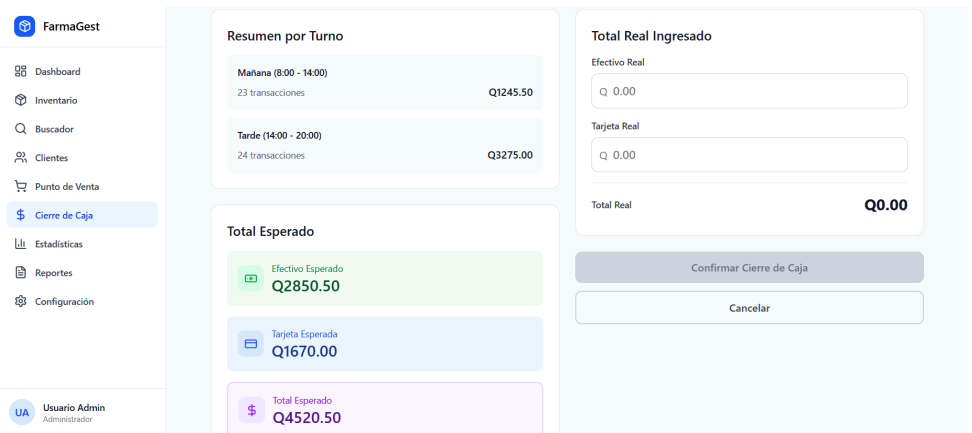
Búsqueda avanzada de productos: un mejor manejo de la búsqueda de medicamentos, incluyendo un autocompletado y diferentes variantes de nombres.



Punto de venta: interfaz simple e intuitiva para la venta de medicamentos.



Cierre de caja óptimo: cero manejo de cuentas a mano y un sistema automático de cierre de caja.



Ingreso de información de productos: ingreso de detalles importantes, como las variantes de nombres, casa farmacéutica, fecha de vencimiento, etc.

Registrar un nuevo medicamento en el inventario

Información Básica

Código del Producto *

MED001

Categoría *

Seleccionar categoría

Nombre Comercial *

Paracetamol 500mg

Nombre Genérico *

Acetaminofén

Proveedor

Farmacéutica XYZ

Datos de Lote

Número de Lote *

LOT2026A

Fecha de Vencimiento *

dd/mm/aaaa

Stock Inicial *

100

Precio y Margen

Costo Unitario *

Q. 0.00

Precio de Venta *

Q. 0.00

Margen de Ganancia

0%

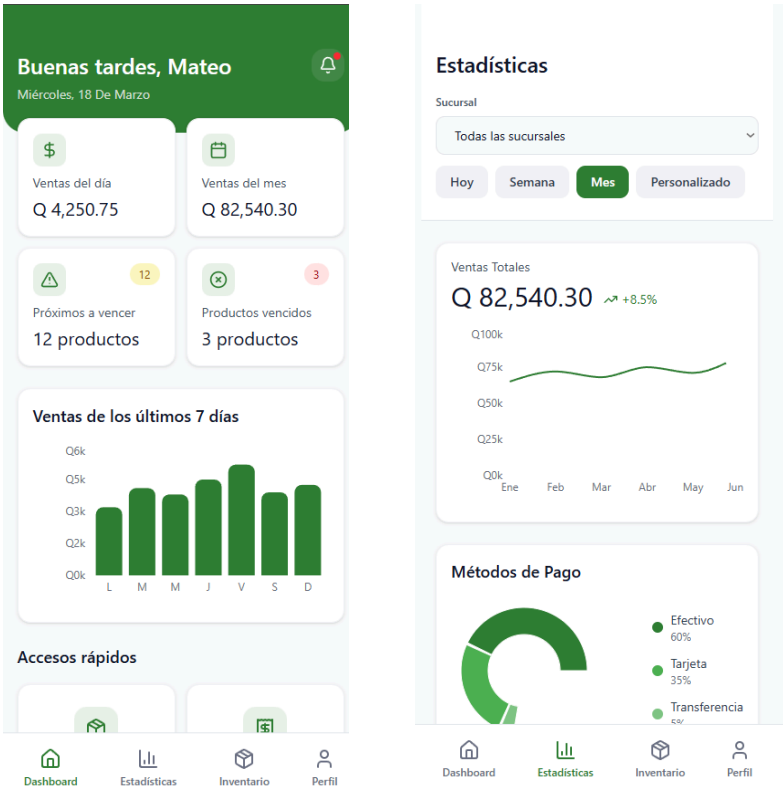
Se calcula automáticamente

Guardar Producto

Cancelar

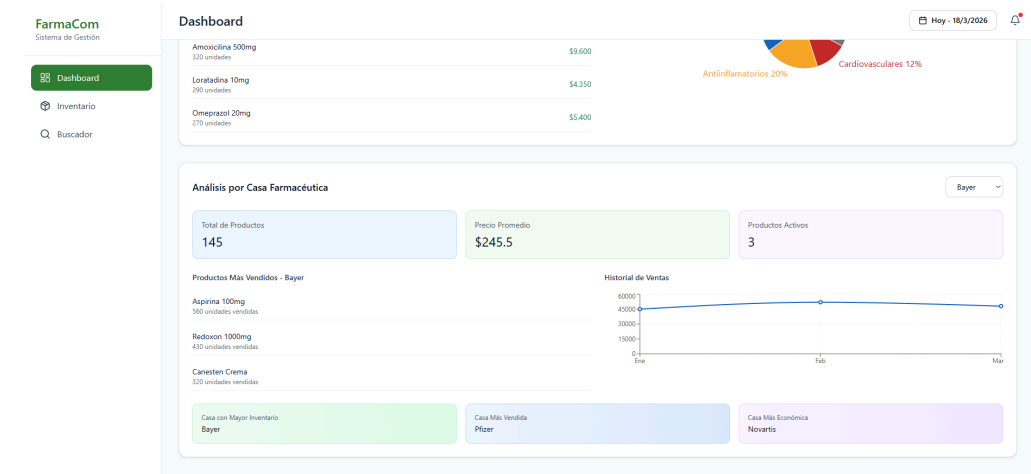
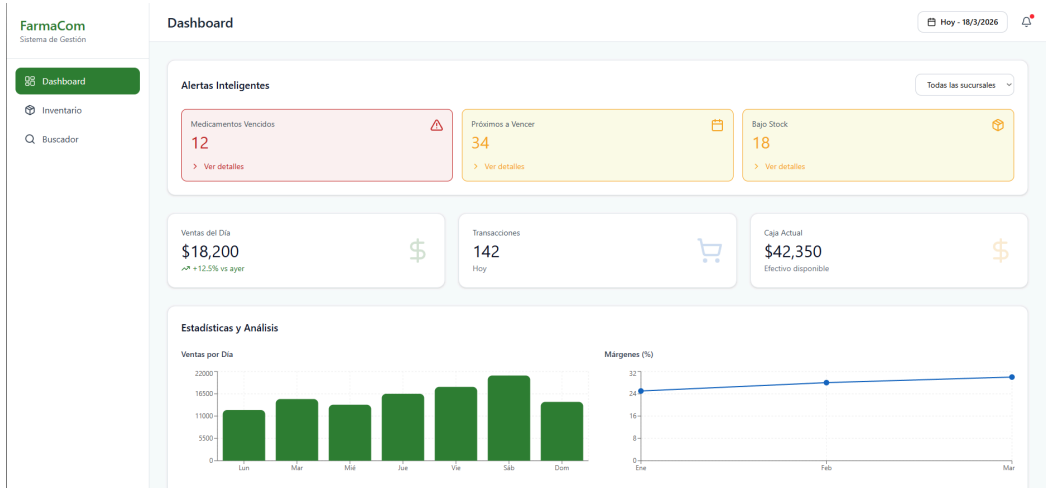
- ii. **Evidencias de cada versión de prototipo**
- En comparación al prototipo v1 del pasado inciso, en esta versión se separaron las funcionalidades según el usuario, tomando en cuenta el punto de venta y el administrador.

Prototipo v2 para Admin en formato móvil: para un control en todo momento y mayor visualización de los datos en tiempo real.



Prototipo v2 para Admin:

Dashboard: unificación del apartado de información preliminar en tiempo real con las estadísticas para un mejor manejo e interfaz mucho más simple. Así mismo incluyendo filtros por sucursales.



Inventario:

FarmaCom
Sistema de Gestión

Dashboard

Inventario

Buscador

Inventario

Hoy - 18/3/2026

Total Productos
13

Vencidos
3

Próximos a Vencer
3

En Buen Estado
7

Q Buscar por nombre, código o genérico...

Todos los estados

Solo Vencidos

Código	Nombre Comercial	Nombre Genérico	Lote	Vencimiento	Stock	Estado	Proveedor
MED-001	Paracetamol	Paracetamol	L2024-01	15/02/2026	45	Vencido	Bayer
MED-002	Ibuprofeno	Ibuprofeno	L2024-02	20/02/2026	30	Vencido	Pfizer
MED-003	Amoxicilina	Amoxicilina	L2024-03	10/03/2026	25	Vencido	Novartis
MED-011	Aspirina	Ácido acetilsalicílico	L2024-11	15/04/2026	60	Próximo a vencer	Bayer
MED-012	Metformina	Metformina HCL	L2024-12	20/04/2026	50	Próximo a vencer	Roche
MED-013	Enalapril	Enalapril Maleato	L2024-13	25/04/2026	40	Próximo a vencer	GSK
MED-021	Diclofenaco	Diclofenaco Sódico	L2024-21	15/06/2026	120	Buen estado	Bayer
MED-022	Captopril	Captopril	L2024-22	20/09/2026	95	Buen estado	Pfizer
MED-023	Ciprofloxacino	Ciprofloxacino HCL	L2024-23	25/10/2026	110	Buen estado	Novartis

Buscador:

FarmaCom
Sistema de Gestión

Dashboard

Inventario

Buscador

Buscador

Hoy - 18/3/2026

Búsqueda Inteligente de Medicamentos

Encuentra rápidamente cualquier producto en el inventario

Q Buscar por nombre comercial, genérico o código...

Filtrar por Categoría

TodasAnalgésicosAntibióticosAntiinflamatoriosCardiovascularesGastrointestinalesAntihistamínicos

Q

Comienza a escribir para buscar medicamentos

Puedes buscar por nombre comercial, genérico o código

Prototipo v2 para las sucursales:

Inventario: eliminación de información solo para admin y cambio en filtros de búsqueda avanzada.

FarmaCom

Inventario

Buscador

Cientes

Punto de Venta

Cierre de Caja

Configuración

Inventario

Control de medicamentos y vencimientos

+ Nuevo Producto

Total Productos8

Vencidos/Criticos2

Próimos a Vencer3

En Buen Estado3

Q. Buscar por código, nombre comercial o genérico...

Todos los estados

Solo Vencidos

Código	Nombre Comercial	Nombre Genérico	Lote	Vencimiento	Stock	Estado	Proveedor
MED001	Paracetamol 500mg	Acetaminofén	LOT2024A	14/3/2024	5	Vencido/Critico	Farmacutíca XYZ
MED002	Ibuprofeno 400mg	Ibuprofeno	LOT2024B	19/5/2026	150	Próximo a vencer	Farmacutíca ABC
MED003	Amoxicilina 500mg	Amoxicilina	LOT2025A	9/8/2027	200	Seguro	Farmacutíca XYZ
MED004	Omeprazol 20mg	Omeprazol	LOT2024C	24/4/2026	10	Próximo a vencer	Farmacutíca DEF
MED005	Losartán 50mg	Losartán potásico	LOT2024D	27/2/2024	3	Vencido/Critico	Farmacutíca ABC
MED006	Metformina 850mg	Metformina	LOT2025B	29/11/2027	180	Seguro	Farmacutíca XYZ
MED007	Loratadina 10mg	Loratadina	LOT2024E	14/6/2026	95	Seguro	Farmacutíca DEF
MED008	Atorvastatina 20mg	Atorvastatina	LOT2024F	9/3/2026	12	Próximo a vencer	Farmacutíca ABC

UA

Usuario Admin

Sucursal San Gabriel

Buscador avanzado:

FarmaCom

Inventario

Buscador

Cientes

Punto de Venta

Cierre de Caja

Configuración

Buscador de Productos

Búsqueda rápida e inteligente

Q. Buscar por código, nombre comercial o genérico...

Filtrar por categoría:

TodosAnalgésicosAntiinflamatoriosAntibióticosGastroenterológicosAntihipertensivosAntidiabéticosAntihistamínicosHipolipemiantes

MED001Vencido/CriticoAgregar al carrito

Paracetamol 500mg

Acetaminofén

LoteLOT2024A

Vencimiento14/3/2024

Stock5 unidades

PrecioQ2.50

MED002Próximo a vencerAgregar al carrito

Ibuprofeno 400mg

Ibuprofeno

LoteLOT2024B

Vencimiento19/5/2026

Stock150 unidades

PrecioQ3.75

MED003SeguroAgregar al carrito

Amoxicilina 500mg

Amoxicilina

LoteLOT2025A

Vencimiento9/8/2027

Stock200 unidades

PrecioQ1.20

UA

Usuario Admin

Sucursal San Gabriel

Cientes:

FarmaCom

Inventario

Buscador

Cientes

Punto de Venta

Cierre de Caja

Configuración

Cientes

Gestión de clientes frecuentes

Buscar por nombre o teléfono...

Maria González

+34 612 345 678

Compras 45

Total gastado Q230.75

Ultima compra 19/2

Juan Pérez

+34 623 456 789

Compras 32

Total gastado Q890.50

Ultima compra 17/2

Ana Martínez

+34 634 567 890

Compras 8

Total gastado Q245.30

Ultima compra 14/1

Carlos Rodríguez

+34 645 678 901

Compras 28

Total gastado Q675.20

Ultima compra 21/2

UA

Usuario Admin

Sucursal San Gabriel

Punto de venta para las sucursales:

Volver a la pantalla principal

Punto de Venta

Busca y agrega productos al carrito

Buscar producto por código o nombre...

Productos Sugeridos

Paracetamol 500mg

Q2.50

Ibuprofeno 400mg

Q3.75

Amoxicilina 500mg

Q8.50

Omeprazol 20mg

Q5.25

Carrito (1)

Ibuprofeno 400mg

Q3.75 c/u

-

1

+

Q3.75

Método de Pago

Efectivo

Tarjeta

Mixto

Efectivo Recibido

0.00

Total

Q3.75

Finalizar Venta

Cierre de caja según horarios de trabajo:

iii. Lista de cambios hechos a cada versión de prototipo

Para el prototipo v1 se inició con cambios en la parte de separación de funcionalidades según usuario que interactúa, ya que se pretende simplificar y optimizar el uso del sistema para cada usuario diferente.

Por otro lado, la parte de filtros en el apartado de “casa farmaceutica” no se había tomado en cuenta dentro del v1, así que se incluyó en el v2, como también la unificación de la parte del dashboard y estadísticas, de modo que fuese más fácil ver toda la información correspondiente en tiempo real.

Así mismo, en el v2 se incluye un filtro de búsqueda por “casa farmacéutica” y poder visualizar la cantidad de productos que se tienen por proveedor, en cambio, el v1 solo incluía estadística de venta y no el historial de precios según el producto y proveedor. Ese tipo de información para el dueño es de vital importancia en la toma de decisiones de compra y venta al futuro y controlar los precios en tiempo real.

Más allá de los cambios en filtros, búsqueda y despliegue de estadística con más filtros, los cambios fueron leves y los requisitos funcionales no se vieron del todo modificados. Esto fue muy revelador, ya que la parte de ideación pasó a ser un punto clave en el armado del primer prototipo que se aproximó mucho a la solución óptima.

iv. Cambios a la lista de requisitos funcionales

1. Filtros para la búsqueda de información, haciendo el proceso mucho más eficaz y permitiendo al usuario tener control de lo que se despliega.
2. Despliegue de estadística básica pero con un control de filtrado por “proveedor” y cantidad de productos que se tiene por “casa farmaceutica”.

3. Se incluyó un histórico de precios según producto, proveedor y casa farmacéutica que se despliega en el dashboard, incluyendo así su información estadística respectiva.

Más allá de cambios en los requisitos funcionales, se agregaron variantes de los requisitos que ya se tenían, solo se debía incluir un poco más de información de ciertos datos y no se cambió por completo o eliminar algún tipo de funcionalidad.

4. Análisis

a. Lista de requisitos funcionales del sistema

i. Enlace de cada requisito funcional a cada historia de usuario

Módulo de gestión de inventario

Requisito Funcional	Historia/s de Usuario Asociada/s
Registro de Productos: El sistema debe permitir el registro de nuevos productos farmacéuticos ingresando: nombre comercial, nombre genérico, casa médica, presentación, precio de compra, precio de venta y fecha de vencimiento.	"Como doctor y dueño de la farmacia, quiero que se lleve un registro de las fechas de vencimiento, para evitar pérdidas con productos caducados." "Como dependiente, quiero que el sistema de la farmacia sea fácil de usar, para evitar errores."
Búsqueda de Productos: El sistema debe permitir la búsqueda de productos por código, nombre comercial o nombre genérico, mostrando resultados en menos de 15 segundos.	"Como enfermera, quiero buscar productos por cualquier nombre (comercial o genérico) , para encontrarlos rápidamente." "Como enfermera, quiero que el sistema tolere errores ortográficos en la búsqueda, para encontrar resultados aunque escriba mal el nombre del medicamento."
Autocompletado en Búsqueda:	"Como dependiente, quiero que el

Mientras el usuario escribe en el campo de búsqueda, el sistema debe sugerir productos que coincidan con el texto ingresado.	sistema muestre autocompletado mientras escribo, para seleccionar productos más rápido."
Tolerancia a Errores Ortográficos: (Pendiente)	"Como enfermera, quiero que el sistema tolere errores ortográficos en la búsqueda, para encontrar resultados aunque escriba mal el nombre del medicamento."
Control de Fechas de vencimiento: El sistema debe permitir registrar la fecha de vencimiento por lote y mostrar una alerta visual, por ejemplo, colores rojo, amarillo, verde, según la proximidad a la fecha de caducidad.	"Como doctor y dueño de la farmacia, quiero que se lleve un registro de las fechas de vencimiento, para evitar pérdidas con productos caducados." "Como enfermera, quiero verificar la fecha de vencimiento de un medicamento antes de venderlo, para garantizar la seguridad del paciente."
Alerta de Stock Mínimo: el sistema debe permitir configurar un stock mínimo por producto y generar una notificación cuando el inventario caiga por debajo de ese límite.	"Como doctor y dueño, quiero recibir alertas de stock mínimo, para reabastecer productos antes de que se agoten y no perder ventas."
Baja de Productos Vencidos: el sistema debe permitir a la administradora marcar un lote como "dado de baja" por vencimiento, eliminándolo del inventario disponible para la venta y registrando la acción.	"Como administradora, quiero poder registrar y dar de baja productos vencidos, para evitar conflictos."
Filtros de Búsqueda Avanzada: El sistema debe permitir filtrar la lista de inventario por categoría, presentación o casa médica para localizar artículos rápidamente.	"Como administradora, quiero filtrar productos por categoría o combinaciones, para localizar rápidamente cualquier artículo."
Duplicar Producto: El sistema debe ofrecer una funcionalidad para "duplicar" un producto existente, facilitando el ingreso de nuevos productos con características similares.	"Como dependiente, quiero poder duplicar productos con características parecidas para hacer más rápido el ingreso de datos".
Historial de Precios: El sistema debe registrar un historial de todos los cambios de precio de un producto, mostrando fecha, precio anterior,	"Como administradora, quiero consultar el historial de cambios de precio de un producto, para decidir ajustes de manera informada."

precio nuevo y usuario que realizó el cambio.	
---	--

Módulo de Gestión de Ventas

Requisito Funcional	Historia/s de Usuario Asociada/s
Registro de Venta: el sistema debe permitir agregar productos a un carrito de compras, calcular el total automáticamente y registrar la venta en la base de datos.	"Como dependiente, quiero poder hacer un cierre de caja obteniendo información del
Procesar Pagos: el sistema debe permitir procesar pagos en efectivo y con tarjeta, pudiendo ser estos de forma separada o combinada para una misma venta.	"Como administradora, quiero que se unifiquen los ingresos de efectivo con los de tarjeta, para facilitar las cuentas."
Generación de Ticket: al finalizar la venta, el sistema debe generar un comprobante que pueda ser impreso o enviado por correo electrónico al cliente.	"Como dependiente, quiero poder imprimir o enviar por correo tickets de venta, para ofrecer opciones de comprobante a los clientes."
Validación de Vencimiento en Venta: Al agregar un producto al carrito, el sistema debe verificar su fecha de vencimiento. Si el producto está vencido, debe bloquear la venta. Si está próximo a vencer, debe mostrar una advertencia visual.	"Como enfermera, quiero verificar la fecha de vencimiento de un medicamento antes de venderlo, para garantizar la seguridad del paciente."
Cierre de Caja: el sistema debe generar un informe de cierre de caja automático por turno o por día, mostrando el total de ventas, el desglose por método de pago y comparándolo con el efectivo esperado.	"Como dependiente, quiero generar cierre de caja automático por turno o día, para entregar cuentas claras al siguiente turno."

Módulo de Gestión de Clientes

Requisito Funcional	Historia/s de Usuario Asociada/s
Registro de Clientes Frecuentes: el sistema debe permitir el alta de clientes frecuentes con datos básicos.	"Como enfermera, quiero registrar un cliente frecuente con sus datos básicos, para poder identificar sus compras habituales."
Historial de Compras por Cliente: al seleccionar un cliente en una venta, el sistema debe mostrar su historial de compras anteriores y los productos que adquiere con frecuencia.	"Como dependiente, al identificar a un cliente frecuente, quiero ver su historial de compras y productos recurrentes, para ofrecer una atención personalizada."

Módulo de Gestión de Reportes y Estadísticas

Requisito Funcional	Historia/s de Usuario Asociada/s
Estadísticas de Ventas: el sistema debe contar con un módulo de estadísticas que muestre, en tiempo real, los ingresos totales diarios, semanales, mensuales y los productos más vendidos.	"Como doctor y dueño de la farmacia, quiero tener estadísticas de venta, para maximizar las ganancias."
Reportes de Rentabilidad: el sistema debe permitir generar informes que muestren ingresos, costos y márgenes de ganancia, pudiendo filtrar por sucursal para comparar el rendimiento.	"Como doctor y dueño, quiero generar informes de rentabilidad por sucursal, para comparar el rendimiento de ambas farmacias."

Módulo de Administración del Sistema

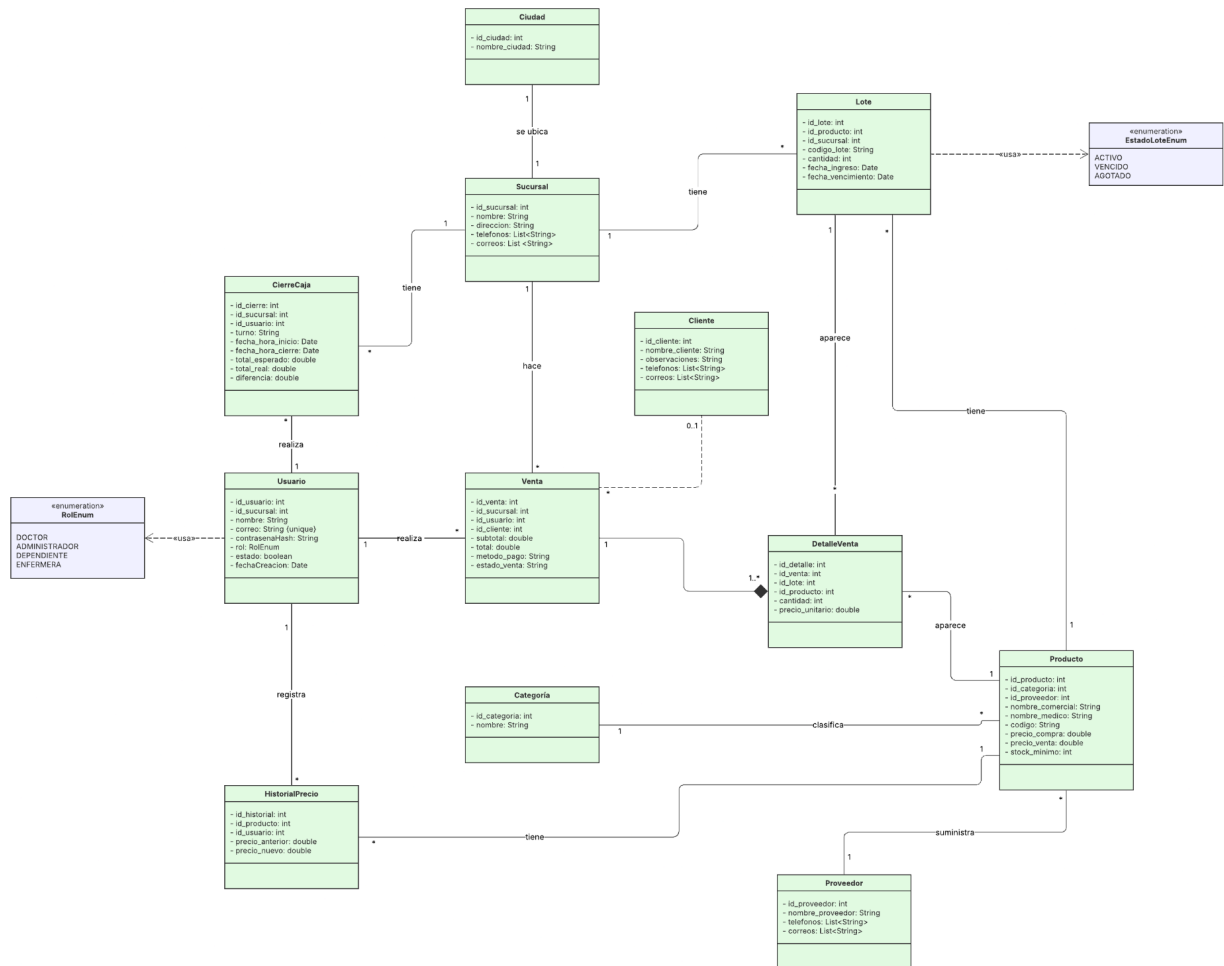
Requisito Funcional	Historia/s de Usuario Asociada/s
Gestión de Usuarios y Roles: el sistema debe permitir al doctor o administrador crear, editar y desactivar usuarios, asignándoles un rol específico: Doctor, Administradora, Dependiente, Enfermera, que	"Como doctor y dueño, quiero poder gestionar usuarios y permisos, para controlar quién accede a cada función del sistema."

determinará sus permisos de acceso.

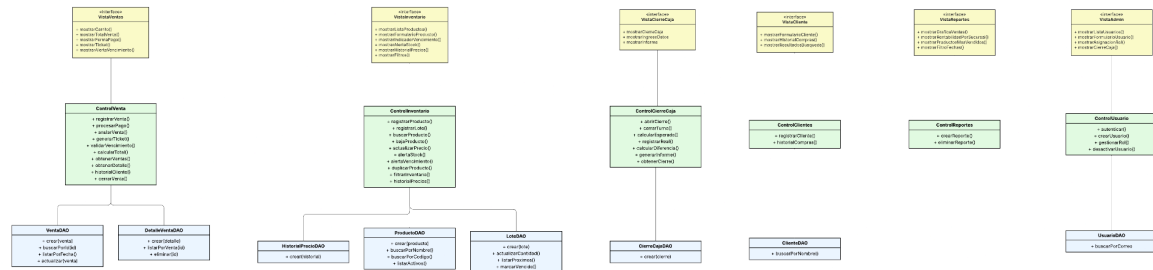
b. Clases preliminares

i. Diagrama de clases

Clases principales



Clases basándose en la arquitectura seleccionada



Link del diagrama para mejor visualización:
https://lucid.app/lucidchart/54fafc02-bd92-4281-9362-4696acf8de23/edit?viewport_loc=1205%2C1140%2C5155%2C2305%2CHWEp-vi-RSFO&invitationId=inv_bc409a41-77bc-48df-81d6-125fb41ff8da

ii. Descripción de las clases

Clases de entidad (modelo)

Sucursal: representa una tienda física de la cadena de farmacias San Gabriel. Es una de las clases estructurales más importantes del sistema porque casi todas las operaciones ocurren en el contexto de una sucursal específica: los lotes de inventario existen dentro de una sucursal, las ventas se registran en una sucursal, los usuarios trabajan en una sucursal, y los cierres de caja se generan por sucursal. Sus atributos de listas teléfonos y correos permiten registrar múltiples datos de contacto sin necesidad de clases auxiliares, ya que en la lógica de negocio un teléfono o correo de sucursal no requiere comportamiento propio. Sin esta clase no sería posible comparar el rendimiento entre las tres tiendas ni saber dónde está físicamente un medicamento.

Ciudad: representa la ciudad donde está ubicada una sucursal. Se extrajo como clase independiente durante el proceso de normalización a Primera Forma Normal para evitar la duplicación del nombre de ciudad en cada registro de sucursal. Su existencia permite que múltiples sucursales pertenezcan a la misma ciudad sin repetir datos, y facilita futuros filtros o agrupaciones de reportes por ciudad.

Usuario: representa a cualquier persona que opera el sistema. Centraliza la identidad, las credenciales y el rol de cada empleado. Es la base del control de acceso del sistema: cada acción registrada, desde una venta hasta un cambio de precio, queda vinculada al usuario que la ejecutó. Su atributo rol de tipo RolEnum determina qué funcionalidades puede ver y usar cada persona, implementando directamente el requisito funcional de gestión de usuarios y roles. El atributo contrasena_hash almacena únicamente el resultado del algoritmo bcrypt, nunca la contraseña en texto plano, cumpliendo el requisito no funcional de cifrado del 100% de las credenciales.

RolEnum: enumeración que define los cuatro roles posibles dentro del sistema: DOCTOR, ADMIN, DEPENDIENTE y ENFERMERA. Centralizar los roles en un enum garantiza que no puedan existir roles no definidos en el sistema, evitando inconsistencias. Cada valor determina un conjunto específico de permisos de acceso.

Producto: es la entidad central del catálogo del sistema. Representa un medicamento u otro tipo de artículo registrado, con sus dos nombres, comercial y médico cuando aplica, su presentación, categoría y proveedor. Contiene los precios de compra y venta para el cálculo de márgenes de rentabilidad, y el atributo stock_minimo que es el umbral para las alertas de reabastecimiento. Es importante distinguir que Producto es el concepto abstracto del medicamento en el catálogo, mientras que Lote representa las unidades físicas reales disponibles en bodega.

Categoría: clasifica los productos en grupos farmacológicos como analgésicos, antibióticos o antiinflamatorios. Se extrajo como clase independiente durante la normalización para evitar duplicación de nombres de categoría en cada producto. Permite los filtros de búsqueda avanzada por categoría y facilita los reportes de distribución de ventas por tipo de medicamento. Cada producto pertenece a exactamente una categoría.

Proveedor: representa la casa médica o laboratorio que suministra un producto. Sus atributos de listas teléfonos y correos permiten registrar múltiples datos de contacto, siguiendo el mismo criterio que en Sucursal. Permite filtrar el inventario por proveedor y es la base para la gestión de reabastecimiento. Cada producto tiene exactamente un proveedor registrado.

Lote: representa un conjunto físico de unidades de un producto que ingresaron juntas a una sucursal en una fecha determinada. Es la clase más crítica para el control farmacéutico porque almacena la fecha de vencimiento específica de ese grupo de unidades, lo que permite implementar FIFO vendiendo primero los lotes que vencen antes, y soporta las alertas de vencimiento por colores definidas en

los requisitos funcionales. Su estado es gestionado mediante EstadoLoteEnum y puede ser ACTIVO, VENCIDO o AGOTADO.

EstadoLoteEnum: enumeración que define los tres estados posibles de un lote. Centralizar estos estados en un enum evita el uso de strings arbitrarios en la base de datos y garantiza que el sistema solo maneje estados válidos. ACTIVO indica que el lote está disponible para la venta, VENCIDO indica que fue dado de baja por caducidad, y AGOTADO indica que sus unidades se consumieron completamente.

HistorialPrecio: registra cada cambio de precio que se realiza sobre un producto, guardando el valor anterior, el valor nuevo, la fecha del cambio y el usuario responsable. Su existencia como clase separada en lugar de un simple campo en Producto es lo que hace posible consultar la evolución completa de precios a lo largo del tiempo. Sirve como respaldo de auditoría financiera y cumple directamente el requisito funcional de historial de precios que permite a la administradora tomar decisiones de ajuste de forma informada.

Venta: representa una transacción comercial completa realizada en el punto de venta. Registra en qué sucursal ocurrió, qué usuario la procesó, a qué cliente se asoció si aplica, el desglose entre efectivo y tarjeta, y el estado de la transacción. Es la clase que integra el proceso de venta de principio a fin y es la fuente principal de datos para los reportes y estadísticas. La relación con Cliente es opcional porque no toda transacción requiere identificar al comprador.

DetalleVenta: representa un renglón individual dentro de una venta, es decir, un producto específico de un lote específico con su cantidad y precio al momento de la transacción. Existe en composición con Venta porque su ciclo de vida depende completamente de ella. Almacenar el precio_unitario en esta clase es intencional: si el precio del producto cambia posteriormente, el registro histórico de la venta permanece intacto, garantizando la integridad de los reportes financieros.

Cliente: representa a un cliente frecuente registrado voluntariamente en el sistema. Sus atributos de listas teléfonos y correos permiten registrar múltiples datos de contacto. Su existencia permite asociar ventas a una persona específica, construir un historial de compras y ofrecer atención personalizada, implementando directamente los requisitos funcionales de registro de clientes frecuentes e historial de compras por cliente.

CierreCaja: resume el movimiento financiero de un turno de trabajo en una sucursal específica. Compara el total esperado según las ventas registradas en el sistema contra el efectivo y tarjeta real contado físicamente al cerrar el turno. La diferencia entre ambos valores es el indicador principal de inconsistencias en

la caja. Implementa directamente el requisito funcional de cierre de caja automático por turno y da visibilidad sobre el rendimiento financiero diario de cada sucursal.

Clases de vistas

VistaVentas: Interfaz responsable de presentar el flujo completo del proceso de venta al dependiente o enfermera. Muestra el carrito de compras, el desglose de totales, las opciones de método de pago y la vista previa del ticket.

VistaInventario: Interfaz que agrupa todas las pantallas de gestión de inventario. Presenta el listado de productos con sus indicadores visuales de vencimiento por color, los formularios de registro y edición, los filtros de búsqueda avanzada y la visualización de lotes individuales.

VistaCierreCaja: Interfaz que presenta el flujo de apertura y cierre de turno. Muestra los datos del cierre en curso, permite ingresar los montos reales contados y presenta el informe final con la diferencia calculada.

VistaClientes: Interfaz que presenta el módulo de clientes frecuentes. Muestra el formulario de registro con datos básicos y el historial de compras por cliente, permitiendo al dependiente identificar productos recurrentes durante el proceso de venta.

VistaReportes: Interfaz que presenta los dashboards de estadísticas y reportes. Muestra gráficas de ventas por período, productos más vendidos, márgenes de ganancia por sucursal y opciones de filtrado por fechas.

VistaAdmin: Interfaz que agrupa las pantallas de administración del sistema. Presenta la gestión de usuarios con asignación de roles y la administración de sucursales, accesible únicamente para los roles DOCTOR y ADMIN.

Clases de controladores

ControlVenta: gestiona toda la lógica de negocio del proceso de venta. Coordina la validación de stock, la verificación de fechas de vencimiento al agregar productos al carrito, el cálculo de totales, el procesamiento de pagos combinados y el registro atómico de la venta junto con sus detalles.

ControlInventario: Controla la lógica del inventario. Maneja el registro de productos y lotes, la búsqueda con tolerancia a filtros, las actualizaciones de precio con registro automático en HistorialPrecio, la baja lógica de productos

vencidos y la emisión de eventos de alerta de stock y vencimiento mediante el patrón Observer.

ControlCierreCaja: Implementa la lógica del cierre de turno. Calcula los totales esperados a partir de las ventas del turno, permite registrar los valores reales contados físicamente y genera el informe de diferencias para la entrega de cuentas entre turnos.

ControlClientes: Administra la lógica de negocio para clientes frecuentes. Gestiona el registro de nuevos clientes y la consulta del historial de compras para ofrecer atención personalizada durante el proceso de venta.

ControlReportes: Contiene la lógica para la generación de reportes y estadísticas. Consulta y agrega datos de ventas para calcular ingresos por período, márgenes de ganancia y productos más vendidos, preparando la información para su visualización en VistaReportes.

ControlUsuario: Gestiona la autenticación y autorización. Valida credenciales de inicio de sesión, genera el JWT con el rol y sucursal del usuario, y administra la creación, modificación y desactivación de cuentas de usuario.

Clases DAO (capa de datos)

VentaDAO: Encapsula todas las operaciones de persistencia sobre la entidad Venta. Provee los métodos necesarios para registrar nuevas ventas, consultarlas por identificador y listarlas por fecha o sucursal para los reportes.

DetalleVentaDAO: Encapsula las operaciones de persistencia sobre DetalleVenta. Gestiona la creación de los renglones individuales de cada venta y su consulta agrupada por transacción.

ProductoDAO: Encapsula las operaciones de persistencia sobre Producto. Implementa las búsquedas por nombre comercial, nombre médico y código, que son la base del requisito funcional de búsqueda en menos de 15 segundos.

LoteDAO: Encapsula las operaciones de persistencia sobre Lote. Provee métodos para consultar lotes próximos a vencer, actualizar cantidades tras una venta y marcar lotes como vencidos, que son las operaciones críticas del control farmacéutico.

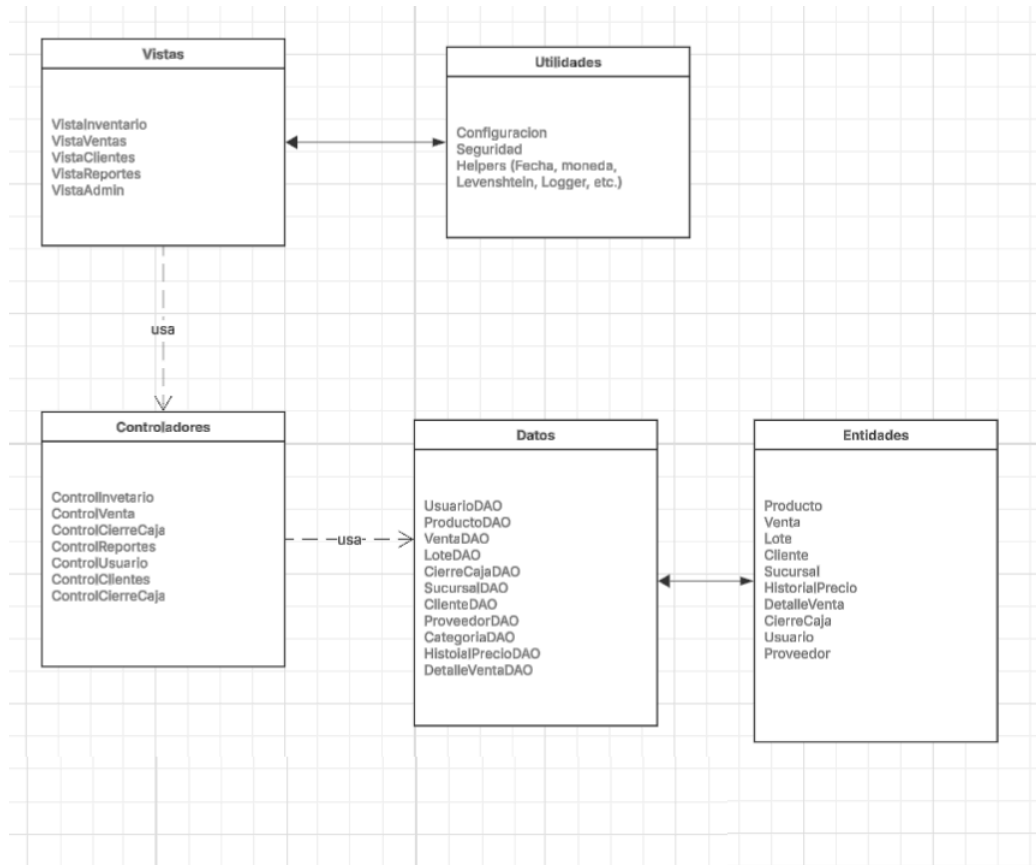
HistorialPrecioDAO: Encapsula las operaciones de persistencia sobre HistorialPrecio. Su método principal registra cada cambio de precio con el usuario responsable y la fecha, siendo invocado automáticamente por ControllInventario cada vez que se modifica el precio de un producto.

ClienteDAO: Encapsula las operaciones de persistencia sobre Cliente. Provee la búsqueda por nombre para la identificación rápida de clientes frecuentes durante el proceso de venta.

CierreCajaDAO: Encapsula las operaciones de persistencia sobre CierreCaja. Gestiona el registro de cada cierre de turno y su consulta para la generación de informes históricos por sucursal.

UsuarioDAO: Encapsula las operaciones de persistencia sobre Usuario. Su método principal de búsqueda por correo es utilizado por ControlUsuario durante el proceso de autenticación para verificar credenciales.

iii. Diagrama de paquetes



Link del diagrama para mejor visualización:

https://lucid.app/lucidchart/ee4d2df3-1ce2-47b5-87dd-ee7edae5ae8f/edit?view_items=U_Epn0PIUUul&page=HWEp-vi-RSFO&invitationId=inv_19689c45-3999-42d8-a12a-4744c01eeaf5

iv. Descripción de cada paquete y sus componentes

Paquete: Vistas (Interfaz)

Este paquete contiene todos los componentes responsables de la interacción con el usuario. La función es mostrar la información y capturar las acciones que el usuario realiza. Sigue los principios de las interfaces gráficas definidas en los prototipos del Design Studio, buscando ser intuitivo y fácil de usar para dependientes, enfermeras, administradora y doctor.

Componentes:

VistaVentas: Pantallas para la gestión de ventas. Incluye la vista del carrito de compras, el panel para procesar pagos en efectivo o tarjeta, la interfaz para aplicar descuentos y la vista previa del ticket.

VistaInventario: Agrupa las interfaces para la gestión de inventario. Aquí se encuentran las pantallas de listado de productos con indicadores visuales de fecha de vencimiento, los formularios para registro y edición de productos, los filtros de búsqueda avanzada y la visualización de lotes individuales.

VistaClientes: Incluye las pantallas para el módulo de clientes frecuentes, como el formulario de registro de clientes con datos básicos y la pantalla de visualización de historial de compras por cliente, que permite al dependiente ver productos recurrentes y ofrecer una atención personalizada.

VistaReportes: Las interfaces para el módulo de reportes y estadísticas. Muestra dashboards con gráficos de ventas diarias/semanales/mensuales, productos más vendidos y márgenes de ganancia por sucursal. Ofrece opciones para filtrar por fechas y exportar reportes a formatos como PDF para la toma de decisiones del doctor.

VistaAdmin: Agrupa las pantallas de administración del sistema, como la gestión de usuarios con asignación de roles, la configuración de permisos basados en roles y la administración de sucursales.

Paquete: Controladores

Encapsula la lógica y las reglas de negocio de la farmacia. Actúa como un intermediario entre la interfaz de usuario y los datos.

Componentes:

ControllInventario: Controla la lógica del inventario. Maneja el registro de nuevos productos y lotes con fechas de vencimiento, la actualización de stocks, las alertas visuales de vencimiento y las notificaciones de stock mínimo para reabastecimiento, la baja lógica de productos vencidos y el historial de cambios de precios.

ControlVenta: Gestiona la lógica de las ventas. Calcula totales automáticamente, valida el stock disponible antes de confirmar la venta, aplica reglas para el pago combinado, verifica fechas de vencimiento al agregar productos al carrito y coordina el registro de la venta y sus detalles.

ControlCierreCaja: Implementa la lógica para el cierre de caja. Calcula los totales esperados por turno/sucursal, permite la conciliación con los valores reales contados físicamente y registra el cierre con su diferencia. Genera el resumen para entregar cuentas claras al siguiente turno.

ControlReportes: Contiene la lógica para generar reportes y estadísticas. Consulta la capa de datos, procesa la información para calcular márgenes de ganancia, totales consolidados por sucursal y productos más vendidos, y prepara los datos agregados para ser mostrados en los gráficos.

ControlUsuario: Gestiona todo lo relacionado con la autenticación y autorización de usuarios. Valida credenciales de inicio de sesión, gestiona sesiones de usuario y verifica los permisos basados en roles para controlar el acceso a las funcionalidades del sistema, garantizando que cada actor sólo acceda a lo que le corresponde.

ControlClientes: Administra la lógica de negocio para los clientes frecuentes, como el registro, la búsqueda por nombre o teléfono y la obtención de su historial de compras para ofrecer atención personalizada, deja ver al dependiente ver productos habituales del cliente durante el proceso de venta.

Paquete: Entidades

Contiene las clases que son el mapa de las tablas de la base de datos relacional. Cada clase representa una entidad del negocio con sus atributos correspondientes.

Componentes:

Usuario: Personal del sistema con rol y permisos.

Producto: Representa cada medicamento.

Venta: Registra cada transacción completa.

Lote: Gestiona las unidades físicas de un producto.

CierreCaja: Resumen financiero por turno.

Sucursal: Información de cada tienda física.

Cliente: Datos de clientes frecuentes.

Proveedor: Casas médicas o laboratorios.

HistorialPrecio: Registro de cada cambio de precio de un producto.

DetalleVenta: Información individual de productos dentro de una venta.

Paquete: Datos

Contiene las clases que encapsulan las consultas a la base de datos, por cada entidad principal existe una clase que implementa las operaciones de acceso a datos.

Componentes:

UsuarioDAO
ProductoDAO
VentaDAO
LoteDAO
CierreCajaDAO
SucursalDAO
ClienteDAO
ProveedorDAO
CategoriaDAO
HistorialPrecioDAO
DetalleVentaDAO
ConexionBD

Paquete: Utilidades

Es un paquete de soporte que contiene clases y componentes de uso común a través de las otras capas. Se promueve la reutilización de código, reduce la redundancia y facilita el mantenimiento al centralizar funcionalidades.

Componentes:

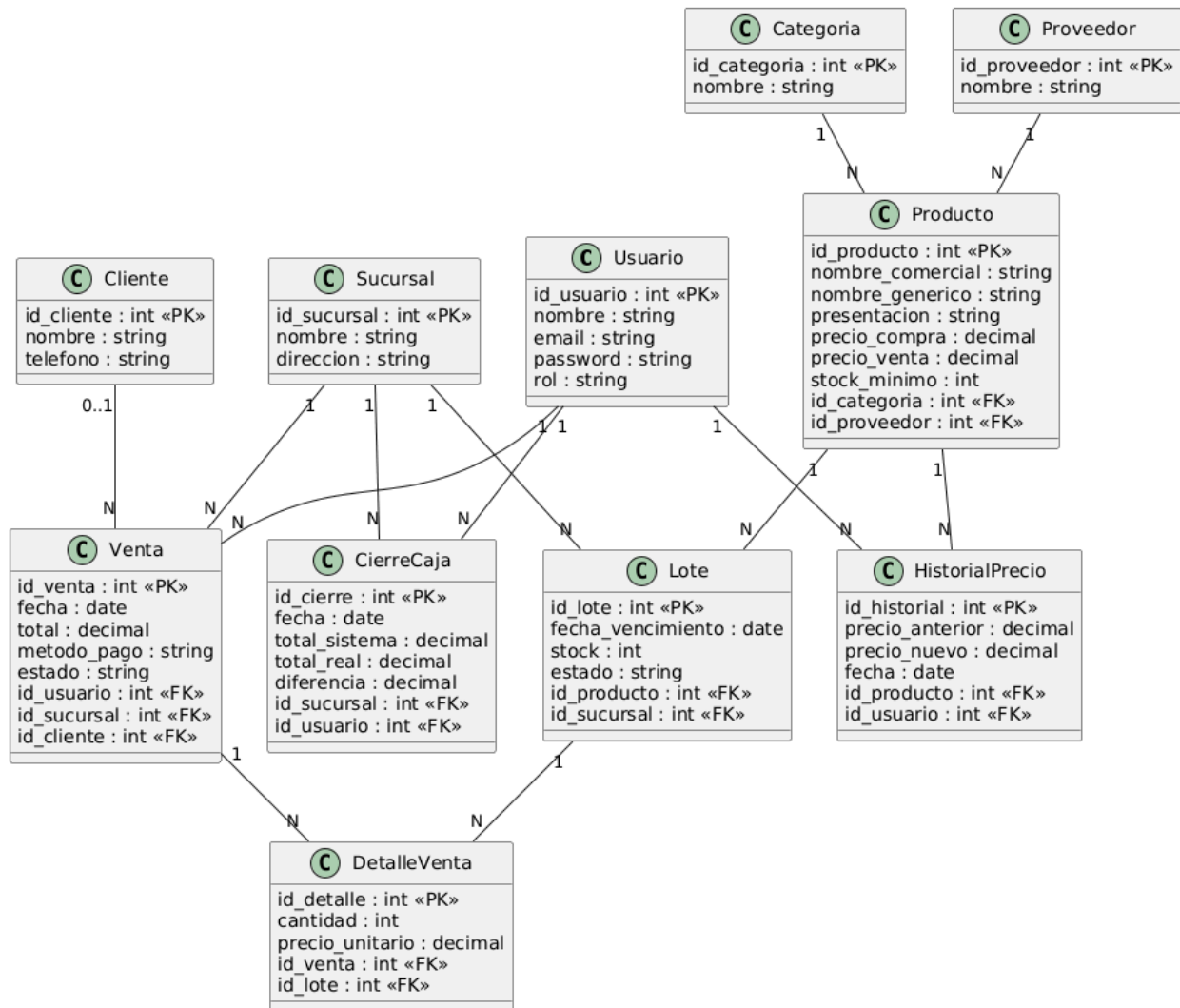
Configuración: Clases para leer y gestionar parámetros de configuración de la aplicación, como la cadena de conexión a la base de datos, tiempos de expiración de sesión, rutas para archivos temporales, umbrales para alertas y stock mínimo por defecto.

Seguridad: Contiene utilidades transversales de seguridad, principalmente funciones para el hashing y verificación de contraseñas, generación de tokens para recuperación de contraseña y validación de sesiones activas.

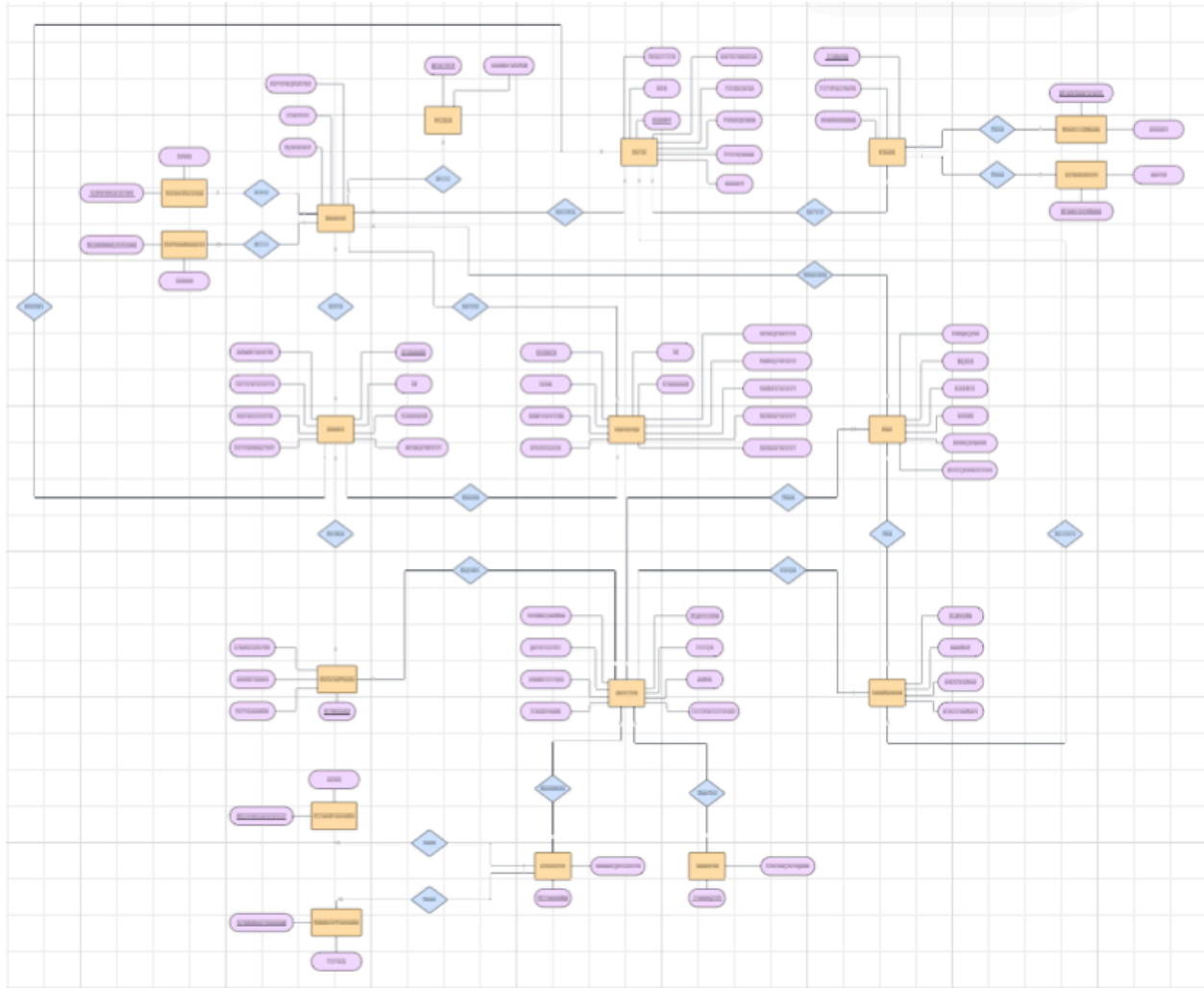
Helpers: Clases con funciones de propósito general y alta cohesión.

c. Persistencia de datos

i. Diagrama de clases persistentes



ii. Diagrama Entidad - Relación de la base de datos relacional



Link del diagrama para mejor visualización:
https://lucid.app/lucidchart/b195675c-03e3-403f-9796-341dafb0d181/edit?view_items=3ekFYVWTQZYi&page=0_0&invitationId=inv_a5b41e27-dbf4-4265-af92-f645bb818a83

5. Diseño

a. Selección de la tecnología a utilizar basándose en los requisitos no funcionales

Requisitos no funcionales

Requisito no funcional	Categoría	Forma en que se medirá su cumplimiento
------------------------	-----------	--

El sistema deberá cumplir con una guía de estilo definida: colores, tipografía y tamaño mínimo de fuente de 12 px, y obtener al menos un 80% de satisfacción en una encuesta de usabilidad aplicada a mínimo 5 usuarios finales después de interactuar con el sistema.	Apariencia o interfaz externa	Evaluación visual por parte de usuarios y docente, verificando legibilidad y consistencia visual.
El 100% de las pantallas del sistema deberán utilizar la misma hoja de estilos definida, misma paleta de colores, tipografía y componentes reutilizables.	Apariencia o interfaz externa	Revisión visual de las pantallas del sistema verificando la consistencia de los elementos gráficos.
El sistema deberá reducir los errores de registro de inventario en al menos un 70% durante el primer mes de uso.	Usabilidad	Comparación de errores antes y después de la implementación del sistema.
El sistema deberá permitir que un dependiente aprenda a usar las funciones básicas en menos de 60 minutos.	Usabilidad	Se seleccionarán dependientes sin experiencia previa en el sistema. Después de una explicación, se les pedirá realizar una venta completa. Se medirá el tiempo desde que inician hasta que finalizan.
El sistema deberá permitir búsquedas de medicamentos en un tiempo máximo de 5 segundos.	Rendimiento	Medición del tiempo de respuesta en búsquedas bajo condiciones normales de uso.
El sistema deberá generar reportes consolidados de ventas en tiempo real con un retraso máximo de 20 segundos.	Rendimiento	Pruebas de generación de reportes midiendo el tiempo de actualización entre sucursales.
El sistema deberá incluir un mecanismo de actualización que permita aplicar correcciones de errores en todas las sucursales con un tiempo de inactividad máximo	Soporte	Simulación de actualización en entorno de prueba con 3 sucursales conectadas; medición del tiempo de inactividad efectivo por sucursal y verificación de que

de 30 minutos.		las transacciones en curso se preserven o se reanuden correctamente.
El sistema deberá permitir mantenimiento y actualizaciones garantizando al 100% que no se pierda ningún tipo de información en el proceso.	Soporte	Se realizará una actualización en un entorno de prueba con una copia de la base de datos real. Antes y después de la actualización se compararán los registros de inventario y ventas para verificar que no haya pérdidas ni alteraciones.
El sistema deberá ser compatible con computadoras que utilicen sistemas operativos Windows.	Portabilidad	Pruebas de instalación y ejecución en equipos con Windows.
El sistema deberá ser accesible a través de un navegador web estándar desde cualquier computadora con conexión a internet dentro de las sucursales.	Portabilidad	Pruebas de funcionamiento en dos computadoras con diferentes sistemas operativos y versiones de navegadores Chrome y Firefox.
El sistema deberá cifrar el 100% de las contraseñas de los usuarios.	Seguridad y privacidad	Inspección de la base de datos de contraseñas y verificación del algoritmo.
El sistema deberá implementar control de acceso basado en roles, permitiendo que solo el administrador y el doctor accedan a datos financieros y de inventario sensibles, mientras que los dependientes solo puedan realizar ventas.	Seguridad y privacidad	Pruebas de seguridad con escenarios de usuario, verificando que el 100% de los intentos de acceso no autorizado sean denegados.
El sistema no deberá incluir expresiones, símbolos o imágenes que puedan resultar ofensivas o contrarias a valores culturales.	Políticos y culturales	Revisión de contenido visual y textual antes de entregar el producto.

El sistema deberá mostrar los valores monetarios en quetzales con dos decimales y las fechas en formato dd/mm/aaaa.	Políticos y culturales	Se revisarán todas las pantallas del sistema que muestren montos y fechas para verificar que cumplen con el formato especificado.
El sistema deberá cumplir con el 100% de la legislación guatemalteca vigente relacionada con el manejo y almacenamiento de información comercial.	Legales	Revisión documental y validación del cumplimiento de normativas legales aplicables.
El sistema deberá garantizar que los registros de ventas puedan ser consultados para fines de auditoría cuando sea requerido por autoridades competentes.	Legales	Verificación de disponibilidad y acceso a reportes históricos de ventas.
El sistema deberá garantizar la integridad y disponibilidad de los datos de inventario y ventas, evitando al 100% las pérdidas o inconsistencias durante su uso.	Confiability	Pruebas de respaldo y recuperación de datos, verificando que la información se mantenga íntegra después de fallos o reinicios del sistema
El sistema deberá mantener un tiempo de disponibilidad mínimo del 99% durante el horario laboral	Confiability	Monitoreo continuo del sistema registrando caídas, tiempos de inactividad y frecuencia de fallos durante el periodo de uso.
El sistema deberá permitir acceder a cualquier módulo principal en un máximo de 3 clics desde el menú principal, verificado mediante pruebas de navegación.	Interfaz interna	Evaluación mediante pruebas de navegación internas verificando consistencia en menús, botones y flujos entre módulos.
El 100% de las acciones críticas como crear, editar, eliminar, cerrar caja, anular venta, deberán mostrar un mensaje de confirmación o error, y al menos el 80% de los usuarios evaluados deberá indicar que entiende el mensaje sin necesidad de explicación adicional.	Interfaz interna	Pruebas de interacción revisando que cada operación muestre retroalimentación adecuada y comprensible para el usuario.

El sistema proporcionará un módulo de ayuda en línea sensible al contexto que explique las funcionalidades de la pantalla actual en un lenguaje claro y sencillo.	Ayudas y documentación en línea	Prueba de la funcionalidad de ayuda en al menos 5 pantallas diferentes del sistema. Se verificará que el contenido mostrado sea relevante para la pantalla desde la que se invocó.
El sistema deberá incluir ayudas contextuales accesibles desde cada módulo principal.	Ayudas y documentación en línea	Revisión funcional verificando que cada pantalla cuenta con íconos o enlaces de ayuda con información relevante.
El sistema deberá funcionar de manera óptima en equipos con al menos 4 GB de memoria RAM y procesador Intel i5 o equivalente.	Hardware	Pruebas de rendimiento del sistema en equipos que cumplan con las especificaciones mínimas.
El sistema deberá operar correctamente en equipos con resolución mínima de pantalla de 1366x768 píxeles.	Hardware	Pruebas de visualización de la interfaz en equipos con la resolución mínima especificada.
El sistema deberá ejecutarse correctamente en computadoras que utilicen el sistema operativo Windows 10 o superior.	Software	Pruebas de instalación y ejecución del sistema en equipos con Windows 10 y versiones posteriores.
El sistema deberá requerir únicamente un navegador web moderno para su funcionamiento, sin necesidad de instalar software adicional.	Software	Verificación del acceso y uso completo del sistema desde navegadores actualizados.
El sistema deberá ser en un diseño modular, separando al menos las funcionalidades de inventario, ventas y reportes en componentes independientes para facilitar el mantenimiento.	Restricciones en el diseño y la implementación	Revisión de arquitectura, confirmando que al menos el 80% de las funcionalidades estén por módulos.
El sistema deberá utilizar exclusivamente tecnologías open source y gratuitas para	Restricciones en el diseño y la implementación	Revisión de la configuración de base de datos y lista de herramientas utilizadas.

su implementación completa, evitando el uso de licencias propietarias pagadas o herramientas comerciales.		
---	--	--

Investigación sobre posibles tecnologías a utilizar

Tecnologías de backend

Node.js (JavaScript + Express)

Descripción: es un entorno de ejecución de JavaScript basado en el motor V8 de Chrome, diseñado para crear aplicaciones de servidor de alto rendimiento. Su arquitectura no bloqueante y basada en eventos permite manejar un gran número de conexiones simultáneas con alta eficiencia, ideal para servicios en tiempo real. Node.js dispone de un amplio ecosistema (npm) con miles de paquetes disponibles, lo que facilita implementar rápidamente funciones como cifrado de contraseñas (bcrypt) y autenticación. Es una plataforma full-stack JS, lo que agiliza el mantenimiento al usar el mismo lenguaje en frontend y backend. Además, su naturaleza modular encaja bien con arquitecturas de microservicios. Aunque el rendimiento bruto de Node.js es muy alto en operaciones de E/S, la gestión de código asíncrono puede ser compleja y presenta una curva de aprendizaje pronunciada para desarrolladores novatos. No obstante, ofrece escalabilidad horizontal sencilla (clusters o contenedores) y múltiples herramientas (como PM2 o Kubernetes) para disponibilidad alta. (Esquivel, 2025).

Ventajas:

- Alto rendimiento y escalabilidad gracias a su modelo asincrónico y no bloqueante.
- Ecosistema maduro (npm) con paquetes para casi cualquier necesidad (autenticación, cifrado, etc.).
- Código JavaScript único en servidor y cliente, facilita mantenimiento y reutilización de módulos.
- Arquitectura modular que permite microservicios independientes, node puede escalar horizontalmente y usar contenedores,.

Desventajas:

- Curva de aprendizaje alta en programación asíncrona y callbacks/promesas.
- Menor rendimiento en operaciones muy intensivas de CPU comparado con lenguajes compilados.
- Eventual complejidad en mantenimiento de código debido a la naturaleza asíncrona (difícil depuración).
- Gestión de dependencias puede volverse compleja en proyectos muy grandes.

Django (Python)

Descripción: Django es un framework de alto nivel basado en Python que promueve el desarrollo ágil y limpio. Incluye de fábrica componentes para autenticación de usuarios, gestión de sesiones, ORM, y protección contra amenazas comunes (XSS, CSRF). Su arquitectura MVC facilita la separación de responsabilidades, lo que mejora la mantenibilidad y escalabilidad de las aplicaciones. Django garantiza un fuerte enfoque en la seguridad, con prácticas seguras por defecto, por ejemplo, hashing de contraseñas con PBKDF2. Es open source y cuenta con una comunidad activa y amplia documentación. En cuanto a rendimiento, Django es competitivo en aplicaciones web de mediano a gran tamaño; puede manejar bien cargas moderadas y escala horizontalmente con herramientas como WSGI, servidores Gunicorn y balanceadores. Para alta disponibilidad, se pueden implementar múltiples instancias en contenedores o servidores, junto con replicación de bases de datos. Django también es compatible con la arquitectura modular, aplicaciones independientes reutilizables, y permite desarrollar APIs que sirvan contenido a apps móviles o PWA, por ejemplo, usando Django REST Framework. Su amplio ecosistema de paquetes facilita integrar nuevas funcionalidades: cifrado avanzado, cacheo, entre otras, aunque el framework es más «monolítico» que otras opciones. Globalmente, Django combina rendimiento razonable con gran seguridad y mantenibilidad (Esquivel, 2025).

Ventajas

- Kit de herramientas «todo en uno»: autenticación, ORM, seguridad, internacionalización, etc. integrados.
- Seguridad robusta: prácticas seguras por defecto, protecciones CSRF, SQL injection evitados automáticamente.
- Excelente escalabilidad estructural: utiliza MVC, fomenta un código organizado y modular.
- Comunidad activa: abundante documentación, tutoriales y librerías de terceros.

Desventajas

- Mayor consumo de recursos de servidor en comparación con frameworks más ligeros.
- Monolítico: menos flexible para arquitecturas extremadamente fragmentadas, aunque soporta microservicios con esfuerzo.
- Migración entre versiones de Django puede requerir ajustes, aunque Django ofrece guías de upgrade.

ASP.NET Core (C#)

Descripción: es un framework web multiplataforma de código abierto desarrollado por Microsoft que permite construir aplicaciones web y APIs con C#. Es gratuito y corre en Windows, Linux y macOS, lo que lo hace versátil. ASP.NET Core extiende .NET con

herramientas para web: middleware, MVC, Razor, Blazor). Tiene excelente rendimiento: en benchmarks de TechEmpower procesa más de 2 millones de respuestas JSON por segundo, superando ampliamente a Node.js y Java puro. Soporta de fábrica características de seguridad: autenticación, autorización, protección XSS/CSRF. Ofrece escalabilidad horizontal mediante despliegue en contenedores o en la nube. Su arquitectura modular, middleware, servicios inyectables, facilita aplicaciones con código organizado y prueba de componentes. Para móviles, puede servir APIs RESTful o integrarse con Blazor Mobile. Además, .NET Core incluye Identity para gestión de usuarios y cifrado de contraseñas seguro, hashing con PBKDF2/BCrypt. La comunidad es amplia y activa en StackOverflow y GitHub. Debido a su rendimiento líder y estabilidad, con soporte a largo plazo, ASP.NET Core es adecuado para sistemas críticos que requieren alta disponibilidad y mantenimiento eficiente (Microsoft, 2026).

Ventajas

- Muy alto rendimiento: líder en benchmarks.
- Multiplataforma y open source, soporta desarrollo en Windows, Linux, Docker.
- Seguridad integrada: autenticación/identidad avanzada, protección XSS/CSRF, MFA, gestión de usuarios robusta.
- Arquitectura modular y orientada a servicios.

Desventajas

- Requiere entorno .NET, que algunos equipos desconocen; menor disponibilidad de desarrolladores .NET en comparación con JS o Python.
- Pesado en recursos en configuraciones pequeñas; mayor memoria base comparado con micro frameworks.
- Históricamente ligado al ecosistema Microsoft, aunque ahora es open source, adopción en pequeñas empresas puede ser menor.

Laravel (PHP)

Descripción: es un framework PHP moderno reconocido por su sintaxis elegante y enfoque en la simplicidad del desarrollador. Ofrece componentes completos, migraciones, ORM Eloquent, colas de trabajo, plantillas Blade, que aceleran el desarrollo de aplicaciones web complejas. Laravel facilita la implementación de seguridad común, hashing de contraseñas con Bcrypt, tokens CSRF, y cuenta con un sistema de paquetes, Composer, que amplía su funcionalidad. Por diseño, es relativamente fácil de aprender para desarrolladores PHP y promueve una arquitectura estructurada con MVC. En términos de rendimiento, Laravel es más lento que Node.js o ASP.NET; en aplicaciones de muy alto tráfico puede requerir optimizaciones, cacheo, Redis, colas asíncronas y escalado horizontal cuidadoso. Sin embargo, es ampliamente utilizado en entornos empresariales con tráfico moderado. Soporta alta disponibilidad mediante balanceo de carga y replicación de bases de datos. Laravel es completamente open source y tiene una gran comunidad. Si se trabaja también con móvil, Laravel puede exponer APIs

RESTful para clientes nativos o híbridos. En conjunto, Laravel es adecuado cuando se prioriza la velocidad de desarrollo y mantenimiento sencillo sobre el rendimiento bruto (Esquivel, 2025).

Ventajas

- Sintaxis y estructura clara que agilizan el desarrollo, curva de aprendizaje suave.
- Completo ecosistema integrado: ORM, migraciones, colas, correos, entre otros, listos para usar.
- Amplia comunidad PHP y documentación detallada; gran repositorio de paquetes (Composer).
- Buen soporte para seguridad común: hashing de contraseñas, protección CSRF, validación.

Desventajas

- Rendimiento subóptimo en aplicaciones de muy alto tráfico, requiere optimización, cachés, optimización de queries.
- Overhead propio de PHP y framework, cada request inicia proceso, menor eficiencia en concurrencia vs. Node o ASP.NET.
- Menor rendimiento de E/S que opciones basadas en JavaScript o C#. Escalar requiere más nodos.
- Dependencia del servidor PHP (FPM) y configuración de hosting, aunque .NET y Node son multiplataforma.

Spring Boot (Java)

Descripción: es un framework de Java para crear aplicaciones independientes rápidamente. Facilita la configuración automática y el desarrollo de microservicios basados en Spring, eliminando buena parte del boilerplate. Soporta todo el ecosistema Spring, Spring Data, Security, Cloud, entre otros, y servidores embebidos, Tomcat, Jetty, simplificando el despliegue. Spring Boot es altamente escalable, su arquitectura y la naturaleza de la JVM permiten aplicaciones que crecen con la demanda (QindeGroup, 2023).. Se usa en entornos empresariales exigentes, garantizando transacciones ACID y facilidad de integración. Para la alta disponibilidad, se combina con técnicas de replicación de base de datos y despliegue en clusters. Spring Boot incluye mecanismos de salud, métricas, y soporta HTTPS y cifrado de datos. En cuanto a mantenimiento, sigue las convenciones de Spring, convenciones sobre configuración, y admite microservicios modulares. Para móvil, puede exponer APIs REST para apps móviles. Su gran comunidad Java asegura soporte y recursos. Aunque el arranque de la JVM es más pesado (latencia de arranque), Spring Boot 3+ mejora el rendimiento con compilación nativa opcional. Es idóneo para sistemas complejos de farmacia que requieran transaccionalidad robusta y buen soporte de mantenimiento (VMware Tanzu, 2026).

Ventajas:

- Simplifica desarrollo Spring: configuración automática y servidores embebidos.
- Gran escalabilidad: la JVM y arquitectura de Spring manejan alta carga y microservicios con facilidad.
- Ecosistema maduro: integración con bases de datos, colas, seguridad, entre otras. Robustez para transacciones.
- Amplio soporte comunitario y empresarial, documentación extensa, Spring Initializr acelera prototipos.

Desventajas:

- Tiempo de inicio y consumo de memoria relativamente altos, JVM, versus entornos nativos.
- Complejidad: gran cantidad de opciones de configuración pueden confundir a nuevos desarrolladores.
- Curva de aprendizaje: dominar Spring completo lleva tiempo.

Tecnologías Frontend

Angular

Descripción: es un framework web frontend desarrollado por Google, basado en TypeScript, orientado a construir aplicaciones SPA (Single-Page Applications) complejas. Proporciona un sólido conjunto de herramientas, CLI, inyección de dependencias, routing, formularios, que permiten desarrollar aplicaciones rápidas y fiables que escalan con el tamaño del equipo y del código. Angular extiende el HTML con componentes reutilizables, data binding de dos vías y un sistema de módulos que favorece la arquitectura modular. Además, soporta servidor-side rendering (SSR) y generación de sitios estáticos (SSG) con hidratación completa del DOM, mejorando SEO y primera carga. Al estar basado en TypeScript, promueve código tipado que aumenta la mantenibilidad. El rendimiento de Angular es bueno para aplicaciones empresariales; con compilación AOT (ahead-of-time) y optimizaciones internas mantiene las UI responsivas incluso en grandes bases de código. Para versiones móviles, Angular se integra con frameworks como Ionic o PWAs, permitiendo experiencias cercanas a nativo. Gracias a su mantenimiento por Google y comunidad activa, dispone de actualizaciones regulares y soporte a largo plazo. Angular cumple requisitos de un sistema de farmacia moderno: permite desarrollo modular, seguridad, sanitización HTML por defecto, y escalabilidad tanto del frontend como para exponer lógica reusable a apps móviles (Angular, 2026).

Ventajas:

- Estructura completa y estandarizada, MVC/MVVM, ideal para aplicaciones de gran envergadura y equipos grandes.

- Performance optimizada: compilación AOT, lazy-loading de módulos, SSR y SSG disponibles.
- Ecosistema profesional: CLI potente, integración con herramientas Angular Material, testing, documentación oficial.
- Fuertes garantías de seguridad, sanitización de templates, políticas de contenido, y mantenimiento continuo, LTS y calendario de releases.

Desventajas

- Curva de aprendizaje relativamente alta, TypeScript + conceptos propios como decoradores, servicios.
- Tamaño de framework grande: mayor volumen de código inicial comparado con librerías ligeras.
- Rigidez en la estructura: puede ser percibido como complejo para proyectos pequeños o prototipos rápidos.

React

Descripción: es una biblioteca de JavaScript creada por Meta (Facebook) para construir interfaces de usuario mediante componentes independientes (React, 2026). No impone patrones estrictos, lo que brinda flexibilidad, y permite combinar componentes escritos por diferentes personas manteniendo consistencia. Su arquitectura modular permite dividir aplicaciones complejas en componentes reutilizables, facilitando el mantenimiento y escalabilidad de grandes proyectos. React utiliza un DOM virtual, en cada cambio de estado actualiza solamente las partes del DOM que cambian, reduciendo los cuellos de botella de rendimiento y manteniendo la UI responsive. Para la gestión de vistas en rutas, se suele complementar con bibliotecas como React Router o frameworks full-stack como Next.js que brindan SSR y generación estática. React es open source y lidera encuestas de popularidad, StackOverflow 2023 lo ubica como el framework web más usado, por lo que hay enorme comunidad y recursos. Para aplicaciones móviles, React aprovecha React Native para compartir lógica y crear apps nativas. React es adecuado para un sistema de farmacia por su alto rendimiento en el frontend, capacidad de escalar, componentes inmutables facilitan trabajo en equipo, y por contar con implementaciones maduras de cifrado y gestión de sesiones en el cliente si se requiere (Atherton, 2025).

Ventajas

- Arquitectura basada en componentes: permite reusar código y facilita el trabajo en equipos grandes.
- Virtual DOM eficiente: actualiza solo lo necesario, obteniendo alta performance aun con interfaces dinámicas.
- Ecosistema maduro: multitud de librerías, estado, enrutamiento, UI, y fuerte soporte comunitario.

- Flexibilidad: se puede integrar gradualmente en proyectos existentes, o usar frameworks full-stack como Next.js para SSR y apps universales.

Desventajas

- Al ser solo UI, requiere combinarlo con otras librerías/frameworks para estado global, routing y build, lo que puede aumentar la complejidad de configuración.
- Curva de aprendizaje de conceptos propios: JSX, hooks, ciclo de vida.
- React cambia con frecuencia, nuevas prácticas como Hooks y Server Components, obligando a equipo a actualizar conocimientos.

Vue.js

Descripción: es un framework progresivo de JavaScript creado para construir interfaces web. Se caracteriza por ser ligero y fácil de aprender, extendiendo HTML con sintaxis declarativa y bindings reactivos. Su sistema de renderizado es verdaderamente reactivo y altamente optimizado por compilación, por lo que ofrece un rendimiento excelente prácticamente sin ajustes manuales. Vue permite iniciar desde una simple capa de vistas, mejorando gradualmente a medida que la app crece, hasta un framework completo con enrutador y gestión de estado. Su ecosistema escalable admite desde pequeños proyectos hasta aplicaciones grandes. La estructura basada en componentes y Single-File Components, componentes en archivos .vue, facilita la modularidad y mantenibilidad. Vue es performant en interfaces SPA gracias a su reactividad granular y puede optimizar automáticamente actualizaciones del DOM. Para mobile, Vue sirve como base de PWAs y se integra con frameworks como Quasar o Ionic Vue para apps nativas. Vue es open source con documentación excelente y comunidad creciente. En resumen, Vue es adecuado para el sistema de farmacias por su balance entre rendimiento y facilidad de desarrollo: agiliza la creación de interfaces responsive y modulares con bajo overhead (Vue.js, 2026).

Ventajas

- Progresivo y versátil, puede ser usado incrementalmente, desde pequeñas mejoras hasta apps completas.
- Alto rendimiento, sistema de render reactivo optimizado, rara vez requiere tuning extra.
- Sintaxis intuitiva y excelente documentación, accesible incluso para desarrolladores sin background profundo en JS.
- Comunidad activa, hay muchas librerías oficiales y plugins disponibles.

Desventajas

- Ecosistema menor que React o Angular, menos componentes listos, aunque crece rápidamente.
- Flexibilidad alta implica menos opiniones de estructura, lo cual puede generar inconsistencia en proyectos grandes si no se imponen convenciones.

- En proyectos muy grandes, puede requerir herramientas adicionales para la arquitectura que no viene de serie.

Tecnologías de base de datos

PostgreSQL

Descripción: es un sistema de gestión de bases de datos relacional-objetual (ORDBMS) open source muy robusto. Almacena datos en tablas con soporte completo ACID por defecto, garantizando integridad aún en fallos. Destaca por su amplio soporte de tipos de datos: JSON, arrays, geometría, XML, etc, y concurrencia multiversión, lo que le permite manejar múltiples lecturas/escrituras simultáneas sin bloquearse (AWS, 2026). PostgreSQL es altamente escalable: soporta particionado, réplicas en modo streaming y arquitecturas maestro-esclavo/hot-standby para alta disponibilidad. Se integra bien con entornos de Kubernetes o cloud para alcanzar $\geq 99\%$ de uptime. Su rendimiento en transacciones complejas es excelente, aunque requiere tuning para cargas extremadamente elevadas. Es open source con fuerte comunidad. En cuanto a móvil, PostgreSQL expone bien APIs mediante frameworks para apps móviles, además de soportar cifrado en reposo opcional (PostgreSQL Global Development Group, 2026).

Ventajas

- Mucha funcionalidad, ACID total, MVCC, rica tipología de datos.
- Escalabilidad y alta disponibilidad integradas, soporta replicación sincrónica/asíncrona y failover.
- Extensible, procedimientos en varios lenguajes, índices avanzados, JSONB para NoSQL híbrido.
- Fuerte comunidad y actualizaciones continuas (licencia BSD, sin costes).

Desventajas

- Configuración y tuning pueden ser complejos, requiere DBA experto para clusters grandes.
- Menos fácil de usar en consultas simples comparado con MySQL, más verboso en SQL avanzado.
- Requiere planificación de hardware/infraestructura para escalado extremo, sharding no nativo.

MySQL/MariaDB

Descripción: MySQL, y su fork MariaDB, es un SGBD relacional muy popular en aplicaciones web. Guarda datos en tablas con SQL tradicional. Es conocido por ser veloz en lecturas, fácil de configurar y ampliamente soportado. Con motores InnoDB ofrece transacciones ACID y bloqueo a nivel de fila, es adecuado para sistemas de mediano a alto volumen de

transacciones. Es open source, la licencia GPL, y cuenta con amplia comunidad. En cuanto a escalabilidad, MySQL soporta replicación maestro-esclavo para alta disponibilidad y puede escalar horizontalmente mediante read-réplicas. Comparado con PostgreSQL, MySQL es más limitado en tipos de datos avanzados y concurrencia, por ejemplo, MVCC solo funciona totalmente con InnoDB. Sin embargo, su simplicidad lo hace eficiente para consultas comunes, ventas, búsquedas rápidas. Con certificación de cifrado en reposo y SSL/TLS en la capa de conexión, cumple requisitos de seguridad (AWS, 2026).

Ventajas

- Amplio uso en web, gran ecosistema de herramientas, hosting y desarrolladores disponibles.
- Buen rendimiento en consultas de lectura intensiva, replicación sencilla para escalabilidad y alta disponibilidad.
- Open source y gratuito, existe versión comunitaria y forks como MariaDB.
- Funciones integradas de respaldo, replicación y control de acceso en el motor.

Desventajas

- Menos características avanzadas que PostgreSQL, tipos de datos, JSONB, vistas materializadas.
- ACID garantizado sólo con motores InnoDB/NDB, otros motores (MyISAM) no lo soportan.
- Concurrencia más limitada, históricamente bloqueos a nivel de tabla en MyISAM, InnoDB mejora pero sin opciones avanzadas.

MongoDB

Descripción: es una base de datos NoSQL orientada a documentos JSON. Está diseñada para manejar grandes volúmenes de datos semi-estructurados, ideal para datos generados por aplicaciones web y móviles. MongoDB destaca en escalabilidad horizontal, permite particionar colecciones entre varios nodos, facilitando ampliar el sistema añadiendo servidores. Su esquema flexible significa que el modelo de datos puede evolucionar sin migraciones rígidas. También ofrece replicación automática y tolerancia a fallos. En términos de rendimiento, es muy rápido en lecturas/escrituras sencillas, gracias a índices B-tree y su naturaleza orientada a documento, aunque tiene menos garantías ACID que las RDBMS tradicionales, hasta versiones recientes solo transacciones a nivel de documento. MongoDB es open source, su licencia SSPL, y posee drivers en múltiples lenguajes (RootStack, 2025).

Ventajas

- Alta escalabilidad horizontal, fácil crecimiento añadiendo nodos.

- Flexibilidad de esquema, no requiere esquema fijo, ágil ante cambios en requisitos de datos.
- Amplio soporte de lenguajes modernos como JavaScript, Python, y APIs JSON naturales.
- Réplicas automáticas y tolerancia a fallos integrada, buen rendimiento en lectura-escritura distribuida.

Desventajas

- Transacciones completas ACID solo recientes, antes de v4.0, no garantizaba integridad entre documentos.
- Consultas complejas como joins no nativas, suelen ser más ineficientes o requerir agregaciones manuales.
- La licencia SSPL impone condiciones, y el ecosistema puede ser complejo.

Apache Cassandra

Descripción: es una base de datos NoSQL de columnas distribuida, diseñada para máxima escalabilidad y alta disponibilidad geográfica. Fue creada para entornos con miles de millones de registros y tráfico masivo. Cassandra no tiene punto único de fallo, todos los nodos son iguales, arquitectura masterless, lo que asegura que si un nodo cae, otros asumen la carga sin interrupción. Brinda escalabilidad masiva, se pueden añadir nodos indefinidamente para aumentar capacidad, sin impacto en la disponibilidad. Cassandra maneja replicación multi-datacenter nativa y consistencia tunable, permite ajustar el balance entre velocidad y coherencia. En lectura/escritura de enormes volúmenes de datos es muy rápida. Sin embargo, su modelo de datos es más rígido, column-family, y las transacciones ACID tradicionales no aplican. Es open source en su versión Apache 2.0 (RootStack, 2025).

Ventajas

- Alta disponibilidad, sin punto único de falla, replicación sincronizada entre nodos garantiza uptime continuo.
- Escalabilidad lineal, escalado horizontal sencillo, ideal para grandes cantidades de datos repartidos en varios centros.
- Modelo de datos optimizado para escrituras masivas en paralelo en múltiples nodos, muy rápido bajo alta carga.

Desventajas

- Consistencia eventual por defecto, las operaciones multi-fila/tables no son ACID y deben modelarse cuidadosamente.
- Complejidad de administración, configuración de anchos de replicación, tuning de compaction.

- Menor ecosistema de herramientas comparado con MongoDB o SQL, curva de aprendizaje en su modelo de datos NoSQL de columnas.

Estimaciones de requisitos no funcionales

Usuarios concurrentes: el sistema opera en 3 sucursales. Considerando que cada sucursal puede tener entre 2 y 4 empleados activos simultáneamente dependientes, enfermeras, administradora, se estima un máximo de 8 usuarios trabajando al mismo tiempo en condiciones normales, con picos de hasta en períodos 12 de alta actividad.

Volumen de transacciones: estimando un promedio de 80 ventas diarias por sucursal, el sistema procesaría aproximadamente 240 ventas por día, 1,680 por semana y 72,000 por año entre las tres sucursales.

Almacenamiento: considerando el volumen de ventas estimado, los registros de lotes, historial de precios y cierres de caja, se proyecta un crecimiento de aproximadamente 500 MB por año de datos transaccionales en PostgreSQL. El sistema debería operar cómodamente dentro de los 5 GB durante los primeros cinco años sin necesidad de particionado.

Tiempo de respuesta: el sistema debe responder búsquedas en menos de 5 segundos y reportes en menos de 20 segundos bajo carga normal de 12 usuarios simultáneos. Estas estimaciones establecen que el sistema es de mediana escala, con carga concurrente baja a moderada, lo que orienta la selección hacia tecnologías que prioricen productividad de desarrollo, seguridad y mantenibilidad sobre rendimiento bruto de alto tráfico.

i. Selección de las tecnologías generales

Backend: Node.js con Express

Node.js opera con un modelo de ejecución no bloqueante y basado en eventos, lo que significa que puede manejar múltiples solicitudes concurrentes sin crear un hilo de sistema operativo por cada una. Para el perfil de carga estimado de 12 a 20 usuarios simultáneos realizando operaciones de ventas, búsquedas de inventario y consultas de reportes, este modelo garantiza holgadamente el cumplimiento del requisito no funcional de búsquedas en menos de 5 segundos y reportes en menos de 20 segundos, dado que la mayoría de las operaciones son intensivas en entrada/salida hacia PostgreSQL, exactamente el escenario donde Node.js tiene mayor ventaja.

El ecosistema npm provee paquetes maduros y auditados para los requisitos no funcionales de seguridad más críticos del sistema. Bcrypt para el hashing de contraseñas cumple directamente el requisito de cifrar el 100% de las credenciales. Jsonwebtoken implementa la autenticación

stateless con JWT necesaria para el control de acceso basado en roles. Express-validator permite la validación centralizada de datos de entrada.

El requisito no funcional de diseño modular, que exige separar inventario, ventas y reportes en componentes independientes, se implementa de forma natural en Node.js mediante módulos de JavaScript. Cada módulo del sistema es un módulo de Node.js independiente, lo que hace que la arquitectura en capas planeada sea directamente expresable en la estructura de archivos del proyecto.

Finalmente, el requisito no funcional que exige usar exclusivamente tecnologías open source y gratuitas se cumple plenamente: Node.js es open source bajo licencia MIT y no tiene costos de licenciamiento en ningún escenario de uso.

Frente a las alternativas evaluadas, Django ofrece buena funcionalidad integrada pero introduce Python como segundo lenguaje cuando ya se está trabajando con JavaScript en el frontend, complicando ligeramente el mantenimiento. ASP.NET Core tiene rendimiento superior en benchmarks, pero su ecosistema históricamente ligado a Microsoft genera dependencia de herramientas que pueden no ser completamente gratuitas en entornos productivos, contradiciendo el requisito de tecnologías open source. Laravel y Spring Boot son sólidos pero agregan complejidad de entorno, PHP y Java respectivamente, innecesaria para el perfil de carga estimado.

Otras herramientas de Backend

Express.js

Es el framework HTTP seleccionado para construir la API REST sobre Node.js. Su diseño permite estructurar el proyecto exactamente según la arquitectura en capas definida, donde cada capa tiene responsabilidades claras y separadas. El sistema de middleware de Express implementa directamente el patrón Middleware Chain definido en los patrones de diseño: cada solicitud HTTP pasa secuencialmente por autenticación JWT, verificación de rol y validación de datos antes de llegar al controlador correspondiente. Esto responde al requisito no funcional de control de acceso basado en roles que exige que el 100% de los intentos de acceso no autorizado sean denegados. Express es open source bajo licencia MIT, cumpliendo el requisito de tecnologías gratuitas, y es el framework más utilizado en el ecosistema Node.js con mantenimiento activo y amplia documentación.

bcrypt

Es la librería seleccionada para el hashing de contraseñas de los usuarios del sistema. A diferencia de algoritmos de hash simples como MD5 o SHA-1, bcrypt está diseñado específicamente para contraseñas: incorpora un factor de costo configurable que hace que el proceso de hashing sea computacionalmente costoso de forma intencional, dificultando ataques de fuerza bruta incluso si la base de datos fuera comprometida. Esto responde

directamente al requisito no funcional de seguridad que exige que el 100% de las contraseñas de los usuarios estén cifradas. Bcrypt es la solución estándar de la industria para este propósito en el ecosistema Node.js.

jsonwebtoken (JWT)

Es la librería seleccionada para implementar la autenticación stateless mediante JSON Web Tokens. Cuando un usuario inicia sesión correctamente, el sistema genera un token firmado que contiene el `id_usuario`, `rol` e `id_sucursal`. Este token viaja en cada solicitud HTTP subsecuente y es verificado por el middleware de autenticación sin necesidad de consultar la base de datos en cada request. Este enfoque responde a dos requisitos simultáneamente: el requisito no funcional de rendimiento, dado que elimina una consulta a base de datos por cada solicitud autenticada, y el requisito funcional de gestión de usuarios y roles, dado que el rol embebido en el token permite al middleware de autorización tomar decisiones de acceso de forma inmediata. JWT también es la base que hace posible la expansión futura a una tienda en línea o aplicación móvil, ya que cualquier cliente puede autenticarse con el mismo mecanismo independientemente de la plataforma.

express-validator

Es la librería seleccionada para la validación de datos de entrada en la capa de controladores. Antes de que cualquier dato llegue a la lógica de negocio o a la base de datos, `express-validator` verifica que los campos requeridos estén presentes, que los tipos de datos sean correctos y que los valores cumplan las restricciones del negocio, por ejemplo que `precio_venta` sea mayor que `precio_compra`, consistente con el CHECK definido en el modelo relacional. Esto responde al requisito no funcional que exige que el 100% de las acciones críticas muestren un mensaje de confirmación o error comprensible para el usuario, dado que los errores de validación son capturados antes de llegar a PostgreSQL y devueltos con mensajes descriptivos. También contribuye al requisito de reducir errores de registro de inventario en al menos un 70% durante el primer mes de uso, previniendo que datos malformados lleguen a la base de datos.

Dotenv

Es la herramienta seleccionada para la gestión de variables de entorno y configuración del sistema. Parámetros como la cadena de conexión a PostgreSQL, la clave secreta para firmar los JWT, el factor de costo de bcrypt y los umbrales de stock mínimo por defecto se almacenan en un archivo `.env` que nunca se incluye en el repositorio de GitHub. Esto responde al requisito no funcional de seguridad de forma transversal: las credenciales de la base de datos y las claves criptográficas no están hardcodeadas en el código fuente, lo que significa que el repositorio puede ser público sin exponer información sensible. Dotenv también implementa la clase Configuración del paquete de Utilidades definida en el diagrama de paquetes.

pg (node-postgres) y pg-pool

pg es el driver oficial de PostgreSQL para Node.js, y pg-pool es su módulo de gestión de pool de conexiones. Juntos implementan el patrón Singleton definido en los patrones de diseño: existe una única instancia del pool de conexiones compartida por todos los DAOs del sistema. pg-pool mantiene un conjunto de conexiones abiertas y reutilizables a PostgreSQL, evitando el costo de establecer una conexión nueva por cada solicitud HTTP. Esto es crítico para cumplir el requisito no funcional de búsquedas en menos de 5 segundos y reportes en menos de 20 segundos bajo carga simultánea de múltiples usuarios en las tres sucursales, dado que el establecimiento de una conexión TCP a PostgreSQL puede tomar entre 20 y 100 milisegundos por sí solo. pg es la librería de acceso a PostgreSQL más utilizada en el ecosistema Node.js, completamente open source y con soporte activo.

EventEmitter (Node.js nativo)

Es el módulo nativo de Node.js seleccionado para implementar el patrón Observer definido en los patrones de diseño. Cuando ControlVenta confirma una venta y actualiza la cantidad de un lote, emite un evento interno que el módulo de alertas escucha de forma completamente desacoplada. Si la cantidad cae por debajo del stock mínimo configurado, se genera la notificación sin que el controlador de ventas tenga conocimiento de esa lógica. Al ser un módulo nativo de Node.js no requiere dependencias adicionales, lo que simplifica el mantenimiento y cumple el requisito de tecnologías open source. Este mecanismo implementa directamente los requisitos funcionales de alerta de stock mínimo y control de fechas de vencimiento con indicadores visuales por color.

Frontend: React

React es la selección para el frontend por razones directamente vinculadas a los requisitos de interfaz del sistema. El requisito no funcional que exige que el 100% de las pantallas utilicen la misma hoja de estilos, paleta de colores y componentes reutilizables se implementa de forma natural mediante la arquitectura de componentes de React. Un componente de botón, tarjeta de producto o indicador de vencimiento por color se define una sola vez y se reutiliza en todos los módulos, garantizando consistencia visual sin esfuerzo adicional. Esto es estructuralmente diferente a un enfoque donde cada pantalla se construye de forma independiente.

El requisito funcional de autocompletado en búsqueda y el de indicadores visuales de vencimiento por colores requieren una interfaz que actualice partes específicas de la pantalla sin recargar la página completa. El DOM virtual de React garantiza que solo los componentes afectados por un cambio de estado se re-rendericen, lo que hace posible una experiencia de búsqueda fluida incluso mientras el usuario escribe, cumpliendo el requisito de respuesta en menos de 5 segundos percibidos.

El requisito no funcional de que cualquier módulo principal sea accesible en máximo 3 clics desde el menú principal se implementa mediante React Router, que permite navegación entre

módulos sin recargar el servidor, haciendo que los cambios de pantalla sean instantáneos desde la perspectiva del usuario.

Para el módulo de reportes y estadísticas, que debe mostrar ingresos diarios, semanales y mensuales en tiempo real, el ecosistema de React provee librerías de visualización maduras como Recharts o Chart.js con bindings nativos para React, reduciendo significativamente el tiempo de desarrollo de ese módulo.

Frente a las alternativas evaluadas, Angular ofrece una estructura rígida que sería beneficiosa para equipos muy grandes, pero para un equipo de cinco personas agrega overhead de configuración que no está justificado por el tamaño del proyecto. Vue.js es una alternativa técnicamente equivalente a React para este caso, pero React tiene una comunidad significativamente más grande, lo que se traduce en más recursos de resolución de problemas disponibles durante el desarrollo del proyecto.

Otras herramientas de frontend

React Router

Es la librería seleccionada para la navegación entre módulos en el frontend. Permite que la transición entre módulos, de ventas a inventario, de inventario a reportes, ocurra sin recargar la página completa desde el servidor, haciendo que los cambios de pantalla sean instantáneos desde la perspectiva del usuario. Esto responde directamente al requisito no funcional de que cualquier módulo principal sea accesible en máximo 3 clics desde el menú principal, dado que la navegación instantánea hace que cada clic sea percibido como inmediato. React Router también permite implementar rutas protegidas por rol: si un dependiente intenta acceder directamente a la URL del módulo de reportes financieros, React Router redirige al usuario sin necesidad de que la solicitud llegue al backend, agregando una capa adicional de control de acceso en el cliente.

Axios

Es la librería seleccionada para realizar las llamadas HTTP desde React hacia la API REST de Node.js. Frente a la alternativa nativa fetch, Axios provee interceptores de solicitud y respuesta que permiten agregar automáticamente el token JWT en el encabezado de autorización de cada solicitud sin repetir ese código en cada llamada. También centraliza el manejo de errores HTTP, de forma que cuando el backend responde con un error 401 de no autorizado o 403 de prohibido, el interceptor puede redirigir automáticamente al usuario a la pantalla de inicio de sesión. Esto implementa de forma consistente el requisito no funcional de que el 100% de las acciones críticas muestren retroalimentación comprensible al usuario, dado que los errores del servidor son capturados y traducidos a mensajes legibles en un único lugar.

React Context + useReducer

React Context combinado con useReducer es la solución seleccionada para el manejo del estado global del frontend. El estado global del sistema incluye el usuario autenticado con su rol y sucursal activa, y el carrito de ventas en construcción. Estos datos deben estar disponibles en múltiples componentes simultáneamente sin pasar props por cada nivel del árbol de componentes. React Context provee ese acceso global, y useReducer organiza las modificaciones al estado mediante acciones predecibles y rastreables. Esta combinación responde al requisito funcional de gestión de usuarios y roles dado que el rol del usuario autenticado, disponible en el contexto global, determina qué opciones del menú son visibles para cada tipo de empleado. Se eligió esta solución sobre Redux porque el volumen de estado global del sistema no justifica la complejidad adicional que Redux introduce.

Recharts

Es la librería seleccionada para la visualización de datos en el módulo de reportes y estadísticas. Está construida nativamente sobre React y SVG, lo que significa que sus gráficos son componentes de React que se integran directamente en la arquitectura de componentes del frontend sin fricciones. Esto responde al requisito funcional de estadísticas de ventas que exige mostrar ingresos diarios, semanales y mensuales y productos más vendidos, y al requisito funcional de reportes de rentabilidad que permite comparar el rendimiento entre sucursales. Recharts es completamente open source bajo licencia MIT y provee los tipos de gráficos necesarios para el sistema: gráficos de barras para comparación entre sucursales, gráficos de línea para tendencias de ventas en el tiempo y gráficos de pastel para distribución por categoría de producto.

TailwindCSS

Es el framework de estilos seleccionado para la capa de presentación. Su enfoque de utilidades predefinidas garantiza consistencia visual de forma estructural: al construir todos los componentes con las mismas clases utilitarias, es imposible que dos pantallas usen tamaños de fuente o colores distintos accidentalmente. Esto responde directamente al requisito no funcional que exige que el 100% de las pantallas utilicen la misma paleta de colores, tipografía y tamaño mínimo de fuente de 12px, dado que esos valores se definen una sola vez en el archivo de configuración de Tailwind y se aplican globalmente. TailwindCSS también facilita el cumplimiento del requisito de resolución mínima de 1366x768 píxeles mediante sus utilidades de diseño responsivo. Es completamente open source bajo licencia MIT.

ii. Elección de la tecnología para almacenamiento persistente

Base de Datos: PostgreSQL

PostgreSQL es la selección para el almacenamiento persistente por razones directamente derivadas del modelo de datos y los requisitos de integridad del sistema. El modelo relacional del sistema, desarrollado y normalizado hasta FNBC, contiene entidades con relaciones complejas e integridad referencial crítica. Una venta no puede existir sin una sucursal y un usuario válidos. Un detalle de venta no puede referenciar un lote que no existe. Un cierre de caja debe garantizar que los totales calculados sean consistentes con los detalles de venta del turno. PostgreSQL garantiza esta integridad mediante soporte ACID completo en todas las configuraciones, a diferencia de MySQL donde las garantías ACID dependen del motor de almacenamiento utilizado. Para un sistema financiero donde el requisito no funcional exige que no se pierda ningún registro de ventas ni inventario, esta garantía es esencial.

Las transacciones de PostgreSQL son críticas para varios flujos del sistema. Cuando se procesa una venta, deben ocurrir de forma atómica: crear el registro en Venta, crear los registros en DetalleVenta, actualizar la cantidad del Lote, y registrar en HistorialPrecio si aplica. Si cualquiera de estas operaciones falla, toda la transacción debe revertirse. PostgreSQL garantiza este comportamiento con sus transacciones ACID, directamente respondiendo al requisito no funcional de integridad y disponibilidad de datos que exige evitar al 100% las inconsistencias.

El requisito funcional de historial de precios y el de reportes de rentabilidad por sucursal requieren consultas analíticas complejas con múltiples joins, agrupaciones y filtros por rango de fechas. PostgreSQL tiene un optimizador de consultas significativamente más sofisticado que MySQL para este tipo de operaciones, y soporta índices parciales y expresiones que permiten optimizar exactamente las búsquedas más frecuentes del sistema, como búsqueda de productos por nombre comercial o genérico.

El volumen proyectado de 5 GB en cinco años está completamente dentro del rango donde PostgreSQL opera con rendimiento óptimo sin configuración avanzada, cumpliendo los tiempos de respuesta requeridos en hardware estándar con los 4 GB de RAM mínimos especificados en los requisitos no funcionales de hardware.

Frente a las alternativas evaluadas, MySQL es una alternativa viable pero inferior para este caso específico por sus limitaciones en consultas analíticas complejas y su garantía ACID condicionada al motor. MongoDB y Cassandra son descartables porque el modelo de datos del sistema es inherentemente relacional, con integridad referencial estricta entre entidades. Forzar ese modelo en una base de datos documental o de columnas introduciría complejidad artificial y perdería las garantías de integridad que el sistema requiere.

Otra herramienta útil: Flyway

Es la herramienta seleccionada para la gestión de migraciones de la base de datos PostgreSQL. Las migraciones son scripts SQL versionados que describen cada cambio al esquema de la base de datos de forma incremental y reproducible. Esto responde directamente al requisito no funcional de mantenimiento que exige que las actualizaciones garanticen al

100% que no se pierda información, dado que Flyway aplica los cambios al esquema de forma controlada y registra qué migraciones ya fueron ejecutadas, previniendo que un script se aplique dos veces o que un cambio se aplique en un orden incorrecto. También garantiza que el esquema de la base de datos en desarrollo, pruebas y producción sea siempre idéntico y reproducible a partir del repositorio de GitHub.

iii. Elección de la arquitectura y los patrones

Selección de arquitectura

El sistema adopta una arquitectura en capas como patrón estructural principal. Esta arquitectura organiza el sistema en capas bien definidas donde cada una tiene una responsabilidad única y solo puede comunicarse con la capa inmediatamente inferior, garantizando bajo acoplamiento y alta cohesión entre componentes.

Capa de Presentación (Vistas): contiene todos los componentes de interfaz de usuario desarrollados en React. Esta capa únicamente se encarga de mostrar información y capturar acciones del usuario, sin contener ninguna lógica de negocio. Esta separación responde directamente al requisito no funcional de consistencia visual, que exige que el 100% de las pantallas utilicen la misma hoja de estilos y componentes reutilizables, algo que solo es garantizable cuando la interfaz está completamente aislada de la lógica.

Capa de Controladores: contiene la lógica de negocio del sistema, implementada en Node.js. Aquí residen los controladores de cada módulo: `ControlInventario`, `ControlVenta`, `ControlCierreCaja`, `ControlReportes`, `ControlUsuario` y `ControlClientes`. Esta capa recibe las solicitudes de la capa de presentación, aplica las reglas del negocio y delega el acceso a datos a la capa inferior. La separación de esta lógica responde al requisito no funcional de diseño modular, que exige que al menos el 80% de las funcionalidades estén organizadas por módulos independientes verificables.

Capa de Datos (DAOs): encapsula todas las consultas a PostgreSQL mediante el patrón Repository. Cada entidad del sistema tiene su clase DAO correspondiente: `ProductoDAO`, `VentaDAO`, `LoteDAO`, entre otras. Ninguna lógica de negocio reside aquí, solo operaciones de acceso a datos. Esto responde al requisito no funcional de mantenimiento, que exige que las actualizaciones no generen pérdida de información, dado que centralizar el acceso a datos en una sola capa permite aplicar y verificar transacciones de forma controlada.

Capa de Entidades: contiene las clases que mapean directamente las tablas de PostgreSQL. Son objetos planos que representan el modelo de datos del negocio.

Capa Transversal de Utilidades: atraviesa todas las capas proveyendo servicios comunes como configuración, seguridad, helpers de fecha y moneda, y logging. Su carácter transversal está

justificado porque funcionalidades como el cifrado de contraseñas o el formato de fechas en dd/mm/aaaa y montos en quetzales con dos decimales, ambos requisitos no funcionales documentados, deben estar disponibles en cualquier capa sin duplicar código.

La elección de esta arquitectura sobre alternativas como microservicios se justifica en base a los requisitos no funcionales. El sistema debe garantizar un 99% de disponibilidad durante el horario laboral y un tiempo de inactividad máximo de 30 minutos durante actualizaciones. Estos umbrales son más fáciles de cumplir y verificar en un monolito modular bien estructurado que en una arquitectura distribuida, donde la coordinación entre servicios introduce puntos adicionales de fallo.

Elección de Patrones de Diseño

Repository Pattern

Implementado en la capa de Datos mediante las clases DAO. Cada clase DAO encapsula todas las operaciones de acceso a datos de su entidad correspondiente, exponiendo métodos como crear(), buscarPorId(), actualizar() y eliminar() sin exponer detalles de las queries SQL subyacentes. Este patrón responde al requisito funcional de búsqueda de productos en menos de 5 segundos, dado que centralizar las queries permite optimizarlas con índices de PostgreSQL sin modificar la lógica de negocio, y también al requisito no funcional de mantenimiento, porque si en algún momento se necesita modificar una query, el cambio ocurre en un único lugar.

Middleware Chain

Implementado en la capa de Controladores de Node.js mediante Express. Cada solicitud HTTP pasa por una cadena de middlewares antes de llegar al controlador correspondiente. El primer middleware verifica y decodifica el JWT del usuario, el segundo verifica que el rol del usuario tenga permiso para acceder a esa ruta específica, y el tercero valida el formato de los datos recibidos. Este patrón implementa directamente el requisito no funcional de seguridad que exige que el 100% de los intentos de acceso no autorizado sean denegados, y el requisito funcional de gestión de usuarios y roles que establece que cada rol tiene acceso diferenciado a las funcionalidades del sistema.

DTO (Data Transfer Object)

Se utilizan objetos de transferencia de datos para definir exactamente qué información viaja entre la capa de presentación y la capa de controladores, y entre la capa de controladores y la capa de datos. Este patrón es crítico para el requisito no funcional de seguridad que exige que el 100% de las contraseñas estén cifradas, porque garantiza que el campo `contrasena_hash` de la entidad Usuario nunca sea incluido en las respuestas que viajan al frontend. También permite que los datos monetarios siempre salgan formateados en quetzales con dos decimales y las fechas en dd/mm/aaaa, cumpliendo el requisito no funcional de formato establecido.

Observer

Se implementa un mecanismo de eventos internos para las notificaciones del sistema. Cuando ControlVenta confirma una venta y actualiza la cantidad de un lote en inventario, emite un evento interno que el módulo de alertas escucha de forma desacoplada. Si la cantidad cae por debajo del stock mínimo configurado, se genera la notificación correspondiente sin que el controlador de ventas tenga conocimiento de esa lógica. Lo mismo aplica cuando se registra un lote nuevo y el sistema debe evaluar colores de vencimiento. Este patrón responde directamente al requisito funcional de alerta de stock mínimo y al de control de fechas de vencimiento con indicadores visuales por color, manteniendo el principio de responsabilidad única en cada controlador.

Factory Method

El módulo de reportes necesita generar distintos tipos de informes: ventas diarias, semanales, mensuales, rentabilidad por sucursal, productos más vendidos. En lugar de tener un único controlador con condicionales para cada tipo de reporte, se utiliza el patrón Factory Method para instanciar el generador de reporte correcto según el tipo solicitado. Esto responde al requisito funcional de estadísticas de ventas y reportes de rentabilidad, y al requisito no funcional de generación de reportes en menos de 20 segundos, porque cada generador puede ser optimizado de forma independiente.

Singleton

La conexión al pool de PostgreSQL se gestiona como un Singleton, garantizando que exista una única instancia del pool de conexiones compartida por todos los DAOs. Esto responde al requisito no funcional de rendimiento, dado que crear una conexión nueva por cada solicitud HTTP sería costoso y haría imposible cumplir el umbral de búsquedas en menos de 5 segundos bajo carga simultánea de múltiples usuarios en las tres sucursales.

6. Informe de gestión

- **Lista de tareas propuestas y responsables**

Tarea	Responsable/s
Hacer resumen, introducción y conclusiones	Jose Abril
Prototipos	Andrés Ismalej
Requisitos funcionales	Todos

Diagrama de clases y su descripción	José Sanchez
Diagrama de paquetes y su descripción	Josué García
Diagrama de clases persistentes	Pablo Orellana
Diagrama Entidad-Relación	Jose Abril
Investigación de posibles tecnologías	José Sanchez
Selección de las tecnologías a utilizar	Todos
Estructura de GitHub	José Sanchez

7. Conclusiones

- Se identificó que la centralización de la información y la generación de reportes en tiempo real fortalecerán la toma de decisiones estratégicas, brindando una visión clara del rendimiento de cada sucursal.
- Se estableció que una arquitectura modular con control de acceso por roles y uso de tecnologías garantizará un sistema seguro, escalable y alineado a las necesidades actuales y futuras del negocio.
- Se determinó que la automatización de procesos reducirá considerablemente los errores en inventario y mejorará la rapidez en tareas críticas como búsquedas de productos, registro de ventas y cierres de caja.
- Se concluye que la implementación de un sistema administrador especializado permitirá optimizar significativamente la eficiencia operativa de Farmacias San Gabriel, al reemplazar procesos manuales y desarticulados por una solución integrada.

Referencias

Amazon Web Services (2026). ¿Cuál es la diferencia entre MySQL y PostgreSQL?. AWS. <https://aws.amazon.com/es/compare/the-difference-between-mysql-vs-postgresql/#:~:text=MySQL%20cuenta%20con%20las%20caracter%C3%ADsticas,ACID%20en%20todas%20las%20configuraciones>

Angular (2026). What is Angular?. <https://angular.dev/overview>

Atherton, D. (2025). Why React Is Still the Best Choice for Building Scalable Web Apps. Dune Technology. <https://www.dunetech.co.uk/post/why-react-is-still-the-best-choice-for-building-scalable-web-apps#:~:text=Scalability>

Esquivel, G., Quisaguano-Collaguazo, L., Caluña-Guaman, A., Llambo-Álvarez, S. (2025). Frameworks del lado del Servidor: Caso de Estudio Node JS, Django y Laravel. 593 Digital Publisher CEIT. https://www.researchgate.net/publication/387907209_Frameworks_del_lado_del_Servidor_Caso_de_Estudio_Node_JS_Django_y_Laravel#:~:text=de%20%20Google%20%20Chrome.caracteriza%20%20por%20%20su

Microsoft (2026). ASP.NET Core. <https://dotnet.microsoft.com/en-us/apps/aspnet#:~:text=ASP>

MIT (2026). The Progressive JavaScript Framework. Vue.js. <https://vuejs.org/>

QindielGroup (2023). Qué es Spring Boot. <https://www.qindiel.com/que-son-los-microservicios-spring-boot/#:~:text=Spring%20Boot%C2%A0es%20un%20framework%20de.la%20complejidad%20de%20la%20configuraci%C3%B3n>

React (2026). React: The library for web and native user interfaces. <https://react.dev/>

Rootstack (2025). The rise of NoSQL databases: What MongoDB, Cassandra and Redis offer. <https://rootstack.com/en/blog/nosql-databases>

The PostgreSQL Global Development Group (2026). PostgreSQL Documentation. PostgreSQL. <https://www.postgresql.org/docs/current/high-availability.html#:~:text=Database%20servers%20can%20work%20together.has%20to%20be%20propagated%20to>

VMware Tanzu (2026). Spring Boot. Spring. <https://spring.io/projects/spring-boot>

Anexo

Formularios LOGT:

Pablo Orellana - 20555						
Fecha	Inicio	Fin	Tiempo de interrupción	Delta tiempo	Fase	Comentarios
08/03/2026	20:00	20:30	0	30 min	quisitos no funciona	Ninguno
10/03/2026	11:45	13:00	0	75 min	Investigación	Ninguno
13/03/2026	22:00	23:00	0	60 min	DER	Ninguno

Andrés Ismalej - 24005						
Fecha	Inicio	Fin	Tiempo de interrupción	Delta tiempo	Fase	Comentarios
10/03/2026	12:30	4:50	120 min	180 min	Prototipado	Creación del prototipo v1 móvil
10/03/2026	7:40	23:50	40 min	240 min	Prototipado	Creación del prototipo v1 escritorio
12/03/2026	8:30	11:00	20 min	70 min	Prototipado	Creación del prototipo v1 móvil parte de estadística
15/03/2026	4:50	7:00	150 min	40 min	Prototipado	Creación del prototipo v2 de escritorio
16/03/2026	10:00	12:00	70 min	50 min	Prototipado	Rediseño del dashboard

Jose Abril - 24585						
Fecha	Inicio	Fin	Tiempo de interrupción	Delta tiempo	Fase	Comentarios
12/03/2026	21:00	22:00	0 min	60 min	DER	Identifiación de identidades y atributos
13/03/2026	11:00	13:00	30 min	90 min	DER	1FN, 2FN, 3FN
15/03/2026	12:00	14:00	15 min	105 min	DER	Diagrama
16/03/2026	21:30	22:00	0 min	30 min	tecnologías	Selección
18/03/2026	11:00	11:40	10 min	30 min	conclusiones	ninguno

